PERSONALISED INTERVIEW PREP

CS Fundamentals Revision Module

Concepts, diagrams, code, dry runs and SQL with outputs

Built for interview revision, not for first-time learning

Shreyansh Paliwal

B.Tech IT, GGSIPU (MAIT)

What is inside

Part A	OOP and C/C++	concepts, vtable, memory layout, pointers, STL, output-prediction dry runs
Part B	Operating Systems	processes, threads, scheduling, sync, deadlock, memory, disk
Part C	Computer Networks	OSI/TCP-IP, IP and subnetting, TCP, DNS, HTTP, TLS, security
Part D	DBMS and SQL	keys, normalisation, transactions, indexing, SQL queries with output

Prepared October 2026. Code samples target C++17 and PostgreSQL syntax unless stated.

Contents

Part A — OOP and C/C++ 4

OOP Concepts 5

C++ Core and STL 10

C Fundamentals and Memory 14

Part B — Operating Systems 18

OS Basics, Processes and Threads 19

CPU Scheduling 22

Synchronisation and Deadlock 24

Memory Management and Virtual Memory 28

File Systems, Disk and I/O 31

Part C — Computer Networks 33

Network Models, Link Layer and Devices 34

IP Addressing, Subnetting and Routing 37

Transport Layer: TCP and UDP 39

Application Layer: DNS, HTTP, TLS and Security 42

Part D — DBMS and SQL 46

DBMS Basics, ER Model and Keys 47

Normalisation and Functional Dependencies 49

Transactions, Concurrency Control and Recovery 51

Indexing, Storage and Query Processing 54

SQL Fundamentals and Sample Schema 56

SQL Joins, Subqueries and Window Functions 59

NoSQL, CAP and Scaling Concepts 64

How to use this module

- **Read in two passes.** Pass 1: read each question and answer aloud before looking at the model answer. Pass 2 (night before): skim Edge cases and Rapid-fire only.
- **Dry runs are the point.** Trace code and schedules by hand first, then compare with the printed output. Interviewers ask "what is the output?" far more than "define X".
- **Code samples are short on purpose.** Each one illustrates one trap. Compile and run the C/C++ ones if an answer surprises you.
- **SQL uses one sample schema** (employees, departments, customers, orders) defined at the start of the SQL topic. Every query shows its output so you can check your reasoning.
- **Not covered here:** DSA problems, system design, language-specific frameworks. These belong in other modules.
- **Suggested schedule:** one topic per day (about 20 topics), then a rapid-fire-only pass for the last two days.

Legend

Q1.	Question and model answer
Edge cases	Gotchas and trick questions interviewers love
Rapid-fire	One-line answers to memorise

Part A — OOP and C/C++

Object-oriented concepts first (examples in C++, with Java/Python differences in a table), then C++ specifics, then C fundamentals and memory. Almost every question here can turn into "what is the output?", so trace the code before reading the answer.

OOP Concepts

> Most asked: the four pillars, polymorphism types, virtual functions, constructors/destructors order, abstract class vs interface, and the diamond problem.

Q1. What are the four pillars of OOP? Give one line and one example each.

- **Encapsulation**: bundle data and the methods that act on it, and hide internals behind access control (`private` fields, public methods). Example: `BankAccount` keeps `balance` private; only `deposit()` and `withdraw()` can change it, so the invariant `balance >= 0` is enforced in one place.
- **Abstraction**: expose *what* an object does, hide *how*. Example: `std::sort` or an abstract `Shape::area()`; the caller does not care about the algorithm or subclass.
- **Inheritance**: a class reuses and extends another (`is-a`). Example: `Dog : Animal`.
- **Polymorphism**: one interface, many behaviours. Compile-time (overloading, templates) and run-time (virtual functions).

Encapsulation vs abstraction: encapsulation is *hiding data* (implementation level); abstraction is *hiding complexity* (design level).

Q2. Class vs object. struct vs class in C++.

A class is a blueprint (type); an object is an instance with its own memory. In C++ the only difference between `struct` and `class` is the **default access**: `struct` members and inheritance are public by default; `class` is private. (In C, `struct` cannot have methods.)

Q3. Explain compile-time vs run-time polymorphism.

Aspect	Compile-time (static)	Run-time (dynamic)
Mechanism	Function/operator overloading, templates	Virtual functions + inheritance
Resolved	By compiler from the static types	At run time through the vtable
Cost	None	One indirection, no inlining
Needs	Different signatures	Same signature, base pointer/reference

```
void print(int x);      void print(double x);   // overloading: chosen at compile time
struct Shape { virtual double area() const = 0; virtual ~Shape() = default; };
struct Circle : Shape { double r; double area() const override { return 3.14159*r*r; } };
Shape* s = new Circle{2.0};  s->area();         // overriding: chosen at run time
```

Q4. Overloading vs overriding vs hiding.

Overloading: same name, different parameter lists, same scope (return type alone cannot distinguish).
Overriding: derived class re-defines a `virtual` function with the *same signature*; chosen at run time.
Hiding: a derived-class function with the same *name* (any signature) hides all base overloads of that name.

```
struct A { void f(int) {}  void f(double) {} };
struct B : A { void f(char) {} };
B b;  b.f(1);        // calls B::f(char): A::f(int) and A::f(double) are hidden!
b.A::f(1);           // OK, explicit. Or add "using A::f;" inside B.
```

Q5. Constructors: types, init order, and destructor order (dry run).

Types: default, parameterised, copy (`T(const T&)`), move (`T(T&&)`). Members are initialised in **declaration order**, not the order in the initializer list. Base class first, then members, then the constructor body. Destruction is the exact reverse.

```
struct A { A(){cout<<"A+ ";} ~A(){cout<<"A- ";} };
struct B : A { B(){cout<<"B+ ";} ~B(){cout<<"B- ";} };
struct C { A a; B b; C(){cout<<"C+ ";} ~C(){cout<<"C- ";} };
int main(){ C c; }
```

Dry run: members first in declaration order. `a` prints `A+`. `b` needs its base first: `A+` then `B+`. Then the body of C: `C+`. Destruction reverses: `C-`, then `b` (`B-` then base `A-`), then `a` (`A-`).

Output: `A+ A+ B+ C+ C- B- A- A-`

Q6. Abstract class vs interface. Compare Java, C++ and Python.

An abstract class has at least one pure virtual function (`= 0`) and cannot be instantiated; it may hold data and implemented methods. An interface is a pure contract with no state.

	C++	Java	Python
Interface	Abstract class with only pure virtuals	`interface` keyword	`abc.ABC` / protocols
Multiple inheritance	Yes (classes)	Classes: no; interfaces: yes	Yes (MRO / C3)
Default method dispatch	Non-virtual unless `virtual`	Virtual unless `final`/`static`/`private`	Always dynamic
Destructor	Deterministic (RAII)	GC, `finalize` unreliable	GC + `__del__`
Access control	`public/protected/private`	Same + package-private	Convention (`_x`, `__x`)

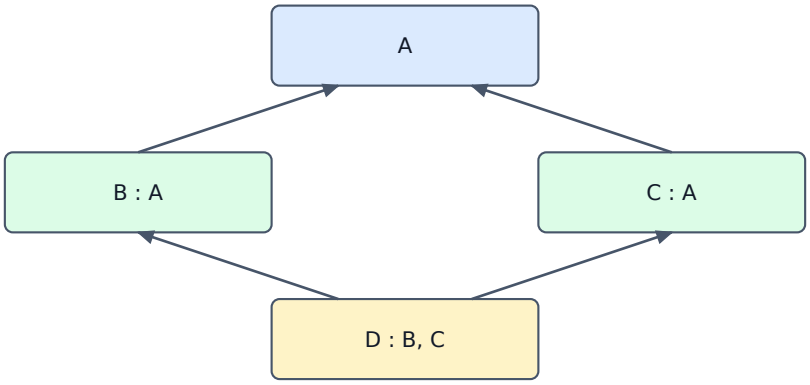
A pure virtual *destructor* is allowed but still needs a body, because derived destructors call it.

Q7. Association, aggregation, composition.

Association: "uses" (Teacher knows Student). Aggregation: "has-a", parts can outlive the whole (Department has Professors). Composition: "owns", parts die with the whole (House has Rooms), usually a member object rather than a pointer. Prefer composition over inheritance when the relationship is not truly "is-a".

Q8. Inheritance types and the diamond problem.

Single, multilevel, multiple, hierarchical, hybrid. **Diamond**: `D` inherits `B` and `C`, both inherit `A`, so `D` contains two copies of `A` and `d.a_member` is ambiguous.

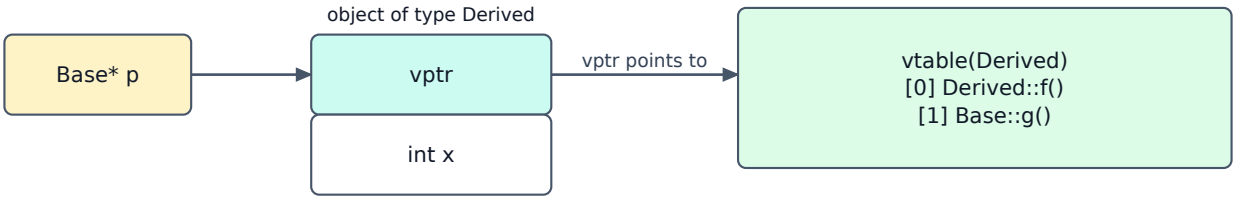


D has two A subobjects unless B, C use virtual A

Fix: **virtual inheritance** (`struct B : virtual A`, `struct C : virtual A`) so only one shared `A` exists; the most-derived class constructs it. Java avoids the problem by disallowing multiple class inheritance; Python resolves it with the C3 method resolution order.

Q9. How do virtual functions work internally? (vptr and vtable)

A class with virtual functions gets a **vtable**: a per-class array of function pointers. Each object stores a hidden **vptr** (usually the first word) pointing to its class's vtable. A call `p->f()` becomes "load vptr, index into the vtable, call".



p->f() == (*p->vptr[0])(p) resolved at run time

Cost: one extra pointer per object, one indirection per call, no inlining across the virtual call.

Q10. Predict the output (virtual dispatch dry runs).

```
struct A { virtual void f(){cout<<"A";}  void g(){cout<<"a";} };
struct B : A { void f() override {cout<<"B";}  void g(){cout<<"b";} };
A* p = new B;   p->f();  p->g();          // (1)
A a = B();      a.f();                    // (2) object slicing
```

(1) `f` is virtual so the dynamic type decides: `B`. `g` is not virtual so the static type `A*` decides: `a`. Output `Ba`.

(2) Copying a `B` into an `A` object **slices** away the B part and the vptr becomes A's: `A`.

```
struct A { virtual void f(int x = 1){cout<<"A"<<x;} };
struct B : A { void f(int x = 2) override {cout<<"B"<<x;} };
A* p = new B;  p->f();                    // prints B1
```

The *function* is chosen dynamically (B::f) but **default arguments are bound statically** from the type `A*` (so `1`). Output `B1`.

```
struct A { A(){ f(); } virtual void f(){cout<<"A";} };
struct B : A { void f() override {cout<<"B";} };
B b;                                      // prints A
```

During `A`'s constructor the object is only an `A` (B is not constructed yet), so virtual dispatch goes to `A::f`. The same applies in destructors. **Never call virtual functions from constructors/destructors.**

Q11. Why must a base class destructor be virtual?

```
struct Base { ~Base(){cout<<"~Base ";} };
struct Derived : Base { int* buf = new int[100]; ~Derived(){ delete[] buf; cout<<"~Derived "; } };
Base* p = new Derived;  delete p;         // only ~Base runs: undefined behaviour, buf leaks
```

Deleting a derived object through a base pointer with a non-virtual destructor is **undefined behaviour** (typically only the base destructor runs). Rule: if a class has any virtual function or is meant to be a base class, make its destructor `virtual` (or `protected` non-virtual).

Q12. static members, `this`, const member functions, friend.

`static` data is shared by all objects (defined once outside the class, `inline static` in C++17); a `static` function has no `this`. `this` is a pointer to the current object (`T* const`). A `const` member function promises not to modify the object (`this` becomes `const T*`) and is the only kind callable on a const object; `mutable` members can still change (caches, mutex). `friend` grants a function/class access to private members, and it is not inherited or transitive; it weakens encapsulation, so use sparingly (e.g. `operator<<`).

Q13. Shallow vs deep copy. What is the Rule of Three / Five / Zero?

The default copy constructor copies members one by one (shallow): two objects end up holding the same raw pointer, so both destructors free it, causing a **double free**.

```
struct Buf { int* d; int n;
  Buf(int n):d(new int[n]),n(n){}
  ~Buf(){ delete[] d; }
  Buf(const Buf& o):d(new int[o.n]),n(o.n){ std::copy(o.d,o.d+n,d); }          // deep copy
  Buf& operator=(Buf o){ std::swap(d,o.d); std::swap(n,o.n); return *this; }   // copy-and-swap
};
```

Rule of Three: if you need a custom destructor, copy constructor or copy assignment, you probably need all three. **Rule of Five** adds move constructor and move assignment. **Rule of Zero**: use `std::vector`, `std::string`, smart pointers so you write none of them.

Q14. Explain the SOLID principles in one line each.

- **S**ingle Responsibility: a class has one reason to change.
- **O**pen/Closed: open for extension, closed for modification (add a new subclass, don't edit a big `switch`).
- **L**iskov Substitution: a subclass must be usable wherever its base is expected without breaking behaviour (a `Square` that breaks `Rectangle`'s width/height contract violates it).
- **I**nterface Segregation: many small interfaces beat one fat one.
- **D**ependency Inversion: depend on abstractions, not concrete classes (inject a `Logger` interface).

Q15. Which design patterns are asked most? Sketch Singleton and Observer.

Creational: Singleton, Factory, Builder. Structural: Adapter, Decorator, Facade, Proxy. Behavioural: Observer, Strategy, Command, Iterator.

```
class Config {                       // Meyers singleton: thread-safe since C++11
  Config() = default;
public:
  Config(const Config&) = delete;   Config& operator=(const Config&) = delete;
  static Config& get(){ static Config inst; return inst; }
};
```

Observer: a subject keeps a list of observers and calls `notify()` on state change (event listeners, pub/sub). **Strategy**: swap algorithms behind one interface (sorting policy, payment method). **Factory**: a function/class decides which concrete subclass to create, so callers depend only on the base type.

Edge cases and gotchas

- A derived function that has the same name as a base function **hides** every base overload (use `using Base::f;`).
- Default arguments on virtual functions are resolved by the static type; avoid them.
- Private inheritance models "implemented-in-terms-of", not "is-a": the base-to-derived conversion is inaccessible.
- A pure virtual function can have a body (call it via `Base::f()`); a pure virtual destructor *must* have one.
- `delete` on a `Derived*` stored in `Base*` without a virtual destructor is undefined behaviour.
- Slicing happens when passing polymorphic objects *by value*; pass by reference/pointer.
- Object members are initialised in declaration order, not initializer-list order (can cause use-before-init bugs).
- Virtual functions cannot be `static`, and constructors cannot be virtual.
- `override` catches signature mistakes at compile time; `final` blocks further overriding.
- Calling a virtual function through an object (not pointer/reference) is resolved statically.

Rapid-fire round

Question	Answer
Four pillars	Encapsulation, abstraction, inheritance, polymorphism
Can a constructor be virtual	No (vptr not set up yet); use a factory / `clone()`

Question	Answer
Can a destructor be virtual	Yes, and it should be in base classes
Size of an empty class	1 byte (distinct addresses)
Pure virtual function	`virtual void f() = 0;` makes the class abstract
Compile-time polymorphism examples	Overloading, templates, operator overloading
Which operators cannot be overloaded	`::`, `.`, `.*`, `?:`, `sizeof`, `typeid`
Composition vs inheritance	has-a vs is-a; prefer composition for flexibility
Why is `friend` controversial	Breaks encapsulation
Where is the vptr set	In the constructor, as each class level is constructed

C++ Core and STL

> Most asked: references vs pointers, new/malloc, smart pointers, move semantics, templates, STL complexities, iterator invalidation, and const-correctness puzzles.

Q1. Pointer vs reference.

	Pointer	Reference
Can be null	Yes	No (must bind to an object)
Reseat to another object	Yes	No, an alias for life
Must be initialised	No	Yes
Arithmetic	Yes	No
Syntax	`*p`, `p->x`	used like the object

References are typically implemented as pointers, but the language guarantees non-null and fixed binding. Use references for parameters that must exist; pointers when "no object" is a valid state.

Q2. Decode `const` placement.

Read right to left.

```
const int* p;        // pointer to const int: *p = 1 is an error, p = &y is fine
int* const p = &x;   // const pointer to int:  *p = 1 is fine, p = &y is an error
const int* const p;  // neither can change
void f(const std::string& s);  // cheap, read-only parameter
```

A `const` member function cannot modify members (except `mutable`). `const` objects can call only const member functions.

Q3. `malloc/free` vs `new/delete`. What about `new[]`/`delete[]`?

`malloc` allocates raw bytes, returns `void*`, returns `NULL` on failure, runs no constructor. `new` allocates and **constructs**, returns a typed pointer, throws `std::bad_alloc` on failure (use `new(std::nothrow)` for null). `free`/`delete` must match the allocator: mixing them, or `delete` on `new[]`, is undefined behaviour.

```
int* a = new int[5];   delete[] a;     // array form must match
Foo* f = new Foo;      delete f;       // calls ~Foo
int* m = (int*)malloc(5*sizeof(int));  free(m);
```

`delete nullptr` is safe. Double `delete` and use-after-free are undefined behaviour.

Q4. What is RAII? Why is it the core C++ idiom?

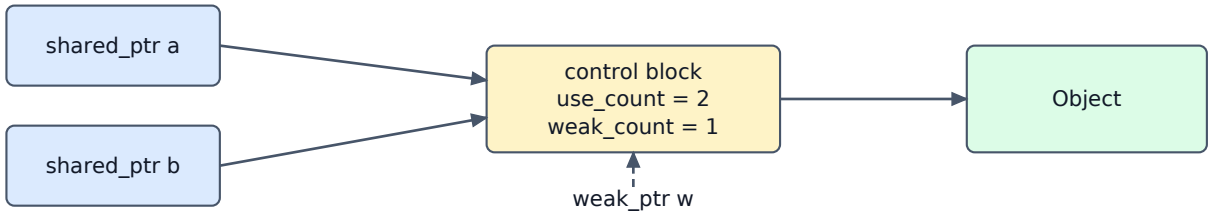
Resource **A**cquisition **I**s **I**nitialisation: tie a resource's lifetime (memory, file, lock, socket) to an object's lifetime: acquire in the constructor, release in the destructor. Because destructors run on every exit path, including exceptions, there are no leaks.

```
{ std::lock_guard<std::mutex> g(m);   // locks
  work();                              // may throw
}                                      // unlocks automatically
```

`unique_ptr`, `shared_ptr`, `vector`, `fstream`, `lock_guard` are all RAII.

Q5. Smart pointers: unique_ptr, shared_ptr, weak_ptr.

- `unique_ptr<T>`: sole ownership, zero overhead, movable not copyable. Default choice.
- `shared_ptr<T>`: shared ownership via an atomic **reference count** in a *control block*; object freed when the count hits 0. Prefer `make_shared` (one allocation).
- `weak_ptr<T>`: non-owning observer; breaks cycles; `lock()` returns a `shared_ptr` or empty.



Cycle leak: `A` holds `shared_ptr<B>` and `B` holds `shared_ptr<A>`, so counts never reach 0. Make one direction `weak_ptr`.

```
auto p = std::make_shared<int>(5);  auto q = p;     // use_count 2
q.reset();                                          // use_count 1
```

Never create two `shared_ptr`s from the same raw pointer (two control blocks, double free).

Q6. lvalue vs rvalue, and move semantics.

An **lvalue** has a name/address (`x`); an **rvalue** is a temporary (`x+1`, `f()`). An rvalue reference `T&&` binds to temporaries, letting a constructor **steal** resources instead of copying.

```
struct Buf { int* d; size_t n;
  Buf(Buf&& o) noexcept : d(o.d), n(o.n) { o.d = nullptr; o.n = 0; }   // steal
  Buf& operator=(Buf&& o) noexcept { if(this!=&o){ delete[] d; d=o.d; n=o.n; o.d=nullptr; o.n=0; }
      return *this; } };
std::string a = "hello";  std::string b = std::move(a);   // a is valid but unspecified (often empty)
```

`std::move` does **not move anything**: it is just a cast to `T&&`. The move constructor does the work. Mark move operations `noexcept` or `vector` will copy on reallocation. `std::forward` preserves value category in templates (perfect forwarding).

Q7. Templates: function/class templates, specialisation, and the cost.

Templates generate code per type at compile time (zero run-time overhead, larger binary, long error messages).

```
template <typename T> T maxOf(T a, T b){ return a > b ? a : b; }
maxOf(3, 7);  maxOf(2.5, 1.0);  maxOf<int>(3, 7.9);   // last: 7.9 converted to int 7
template <> const char* maxOf(const char* a, const char* b){ return strcmp(a,b)>0 ? a : b; }  //
    specialisation
```

Templates are defined in headers (the compiler needs the body to instantiate). `typename` vs `class` are equivalent in template parameters. C++20 adds **concepts** to constrain templates.

Q8. STL containers: underlying structure and complexity.

Container	Structure	Access	Insert/erase	Notes
`vector`	dynamic array	O(1)	end: amortised O(1); middle: O(n)	contiguous, cache-friendly
`deque`	chunked array	O(1)	both ends O(1)	not contiguous
`list`	doubly linked	O(n)	O(1) given iterator	poor cache locality
`map`/`set`	red-black tree	O(log n)	O(log n)	ordered, stable iterators
`unordered_map/set`	hash table	O(1) avg, O(n) worst	O(1) avg	no order; rehash invalidates iterators
`priority_queue`	binary heap on vector	top O(1)	push/pop O(log n)	max-heap by default
`stack/queue`	adapters on deque		O(1)	

`vector` growth is geometric (usually x2 or x1.5), which gives amortised O(1) `push_back`. `reserve(n)` avoids reallocations; `size()` is element count, `capacity()` is allocated slots.

Q9. Iterator invalidation: when does it bite? (dry run)

```
vector<int> v = {1,2,3,4};
for (auto it = v.begin(); it != v.end(); ++it)
    if (*it == 2) v.erase(it);              // BUG: it is invalid after erase
```

Correct: `it = v.erase(it);` (returns the next valid iterator) and only `++it` when not erasing. `push_back` may reallocate and invalidate **all** iterators/pointers/references; `insert`/`erase` in the middle invalidates from the point of change. For `map`/`list`, only the erased element's iterator is invalidated. Erase-remove idiom: `v.erase(remove(v.begin(), v.end(), x), v.end());`.

Another classic: `map[key]` **inserts** a default value if the key is missing; use `find`/`count`/`at` for lookups.

Q10. Lambdas and capture.

```
int x = 10;
auto byVal = [x](int a){ return a + x; };       // copy of x at creation
auto byRef = [&x](int a){ return a + x; };      // sees later changes, dangles if x dies
x = 20;   byVal(1);   // 11
          byRef(1);   // 21
sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
```

`[=]` captures all used locals by value, `[&]` by reference; `mutable` lets a by-value copy be modified inside. Returning a lambda that captures locals by reference is a dangling-reference bug.

Q11. C++ casts.

`static_cast` (checked at compile time: numeric conversions, up-casts, known down-casts), `dynamic_cast` (safe down-cast using RTTI; returns `nullptr` for pointers / throws `bad_cast` for references; needs a polymorphic type), `const_cast` (add/remove `const`; modifying a truly const object is UB), `reinterpret_cast` (reinterpret bits, e.g. pointer to integer, most dangerous). Avoid C-style casts: they silently pick any of the above.

Q12. Exceptions and exception safety.

`throw` unwinds the stack, destroying locals (RAII makes this safe). Catch by **const reference** (`catch (const std::exception& e)`) to avoid slicing. Never let an exception escape a destructor (it terminates if another is active). Guarantees: *basic* (no leaks, valid state), *strong* (rollback: copy-and-swap), *nothrow* (`noexcept`). Constructors can throw; a partially constructed object's *completed* members are destroyed, its destructor is not run.

Q13. `inline`, `static`, `extern`, `constexpr`, `volatile`.

`inline` permits definitions in headers (ODR-safe) and hints at inlining. File-scope `static` gives internal linkage; a function-local `static` lives for the program and is initialised once (thread-safe in C++11). `extern` declares a variable defined elsewhere. `constexpr` allows compile-time evaluation. `volatile` tells the compiler a value may change outside the program's view (hardware registers), so no caching in registers; it does **not** make operations atomic or thread-safe (use `std::atomic`).

Q14. What is undefined behaviour? Give typical examples.

The standard places no requirements on the program, so anything may happen, including "works on my machine". Examples: out-of-bounds access, null dereference, use-after-free, signed overflow (`INT_MAX + 1`), reading an uninitialised variable, data races, modifying a string literal, unsequenced modifications (`i = i++ + ++i`), shifting by >= bit width, missing `return` in a non-void function. Tools: `-Wall -Wextra`, AddressSanitizer, UBSan, Valgrind.

Edge cases and gotchas

- `vector<bool>` is a special bit-packed container: `auto x = v[0]` is a proxy, not a `bool&`.
- Reference to a vector element dangles after `push_back` reallocates.
- `std::map` with `operator[]` on a const map does not compile (it may insert).
- Strings: `std::string` is not guaranteed null-terminated via `data()` before C++11; use `c_str()` for C APIs.

- Comparing signed with unsigned (`-1 < 1u` is false) converts the signed value.
- `auto` drops references/const: use `const auto&` to avoid copies in range-for.
- `sizeof` on a class includes padding and the vptr; empty-base optimisation exists.
- A moved-from object is valid but unspecified: only destroy or assign it.
- `shared_ptr` count updates are atomic, but the *pointed-to object* is not thread-safe.
- Pass by value then `std::move` is a good pattern for "sink" parameters.

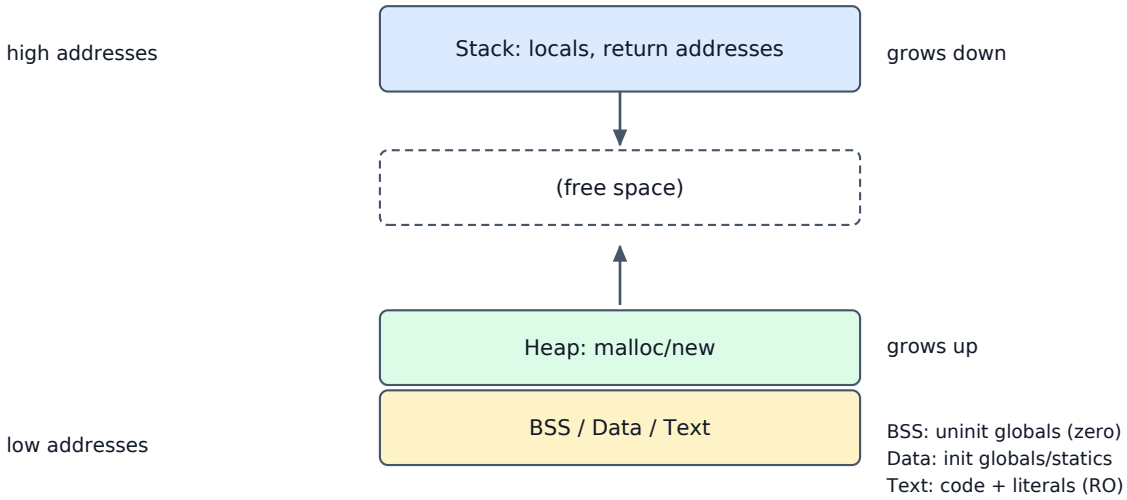
Rapid-fire round

Question	Answer
Pass large read-only object	`const T&`
Default smart pointer	`unique_ptr`
`nullptr` vs `NULL`	Typed null pointer constant vs macro that is `0`/`(void*)0`
`vector` vs `list` default	`vector` almost always
`map` vs `unordered_map`	Ordered O(log n) vs unordered O(1) average
Does `std::move` move	No, it casts to rvalue reference
`new` fails	Throws `std::bad_alloc`
`explicit` keyword	Blocks implicit conversions via single-argument constructors
`= delete` / `= default`	Remove / request compiler-generated special member
`auto` / `decltype`	Deduce type from initialiser / from an expression

C Fundamentals and Memory

Most asked: memory layout, pointers and arrays, struct padding, dynamic memory bugs, output-prediction puzzles, bit manipulation, and function pointers.

Q1. Describe the memory layout of a C/C++ program.



- **Text/code**: machine code, read-only; string literals usually live in read-only data.
- **Data**: initialised globals and statics. **BSS**: uninitialised globals/statics, zero-filled.
- **Heap**: dynamic memory (`malloc`, `new`), managed by you, can fragment.
- **Stack**: one frame per function call (locals, arguments, return address, saved registers); freed automatically; limited size (typically 1 to 8 MB), so deep recursion gives **stack overflow**.

```
int g = 5;                // data
int z;                    // bss
static int s;             // bss
const char* lit = "hi";   // pointer in data, "hi" itself in read-only memory
void f(){ int a;          // a: stack
          int* h = malloc(4); }   // h on stack, the 4 bytes on heap
```

Q2. Pointers and arrays: what is the relationship? (dry run)

An array name *decays* to a pointer to its first element in most expressions, but it is not a pointer: `sizeof` and `&` behave differently. `a[i]` is defined as `*(a+i)`; pointer arithmetic scales by element size.

```
int a[5] = {10,20,30,40,50};   int* p = a;
printf("%zu %zu\n", sizeof(a), sizeof(p));   // 20 8   (64-bit)
printf("%d %d\n", *(p+2), 2[a]);             // 30 30  (a[2] is *(a+2) is *(2+a) is 2[a])
printf("%d\n", *p++);                        // 10, then p moves to a[1]
printf("%d\n", (*p)++);                      // 20, a[1] becomes 21
printf("%d\n", *++p);                        // p moves to a[2], prints 30
```

Passing an array to a function passes a pointer, so inside the function `sizeof(param)` is the pointer size, not the array size:

```
void f(int arr[]){ printf("%zu", sizeof(arr)); }   // prints 8, not 20
```

`&a` has type `int(*)[5]`: `&a + 1` jumps over the whole array (20 bytes), while `a + 1` jumps one element (4 bytes).

Q3. Pointer to pointer, function pointers, `void*`.

```
int x = 5;  int* p = &x;  int** pp = &p;   **pp = 9;     // x is 9
int add(int a, int b){ return a+b; }
int (*fp)(int,int) = add;   fp(2,3);                      // 5   (callbacks, qsort, dispatch tables)
int cmp(const void* a, const void* b){ return *(int*)a - *(int*)b; }
qsort(arr, n, sizeof(int), cmp);   // careful: the subtraction can overflow; prefer (a>b)-(a<b)
```

`void*` can hold any object pointer but cannot be dereferenced or used in arithmetic (a GCC extension aside). In C it converts implicitly; in C++ it needs a cast.

Q4. Strings in C: arrays vs literals, and the classic bugs.

```
char a[] = "hello";       // modifiable copy on the stack, sizeof 6 (includes '\0')
char* s  = "hello";       // points to a literal: s[0] = 'H' is UNDEFINED (often segfault)
printf("%zu %zu", strlen(a), sizeof(a));       // 5 6
char dst[5];  strcpy(dst, "hello");            // overflow: needs 6 bytes
```

Use `strncpy` carefully (may not null-terminate), `snprintf`, or `strlcpy`. `strlen` is O(n) and excludes `'\0'`. `strcmp` returns 0 on equality, so `if (strcmp(a,b))` means "different". Comparing with `==` compares addresses.

Q5. Struct padding and alignment (dry run).

The compiler aligns each member to a multiple of its size and pads the struct so that arrays of it stay aligned.

```
struct A { char c; int i; char d; };      // c@0, pad 3, i@4, d@8, pad 3  => sizeof 12
struct B { int i; char c; char d; };      // i@0, c@4, d@5, pad 2         => sizeof 8
struct __attribute__((packed)) P { char c; int i; };   // sizeof 5, but unaligned access
```

Order members largest-to-smallest to reduce padding. `offsetof(struct A, i)` is 4. A `union` shares one memory block among members: size is the largest member.

```
union U { int i; char c[4]; };   U u; u.i = 0x01020304; printf("%d", u.c[0]);   // 4 on little-endian
```

Endianness: little-endian stores the least significant byte at the lowest address (x86, ARM usually). Network byte order is big-endian (`htons`, `htonl`).

Q6. Dynamic memory bugs: list and examples.

Bug	Example	Result
Memory leak	`p = malloc(10); p = NULL;`	memory unreachable
Double free	`free(p); free(p);`	heap corruption
Use after free	`free(p); *p = 1;`	UB, subtle corruption
Dangling pointer	return address of a local	UB
Buffer overflow	`a[10]` on `int a[10]`	stack/heap smash, security hole
Uninitialised read	`int x; printf("%d", x);`	garbage / UB
Freeing non-heap	`free(&x)`	crash

```
int* bad(){ int x = 5; return &x; }     // returns address of dead stack frame
```

`malloc` can return `NULL`; always check. `calloc(n, size)` zero-fills; `realloc` may move the block (assign to a temp first so you don't lose the original on failure). `free(NULL)` is safe. Set pointers to `NULL` after `free`.

Q7. Predict the output (C puzzle set).

```
int i = 5;  printf("%d %d", i++, i++);        // unspecified order of evaluation: do not rely on it
int j = 5;  int y = j++ + ++j;                // undefined behaviour (unsequenced modification)
printf("%d", sizeof('a'));                    // 4 in C (char literal is int), 1 in C++
if (-1 < 0u) puts("yes"); else puts("no");    // "no": -1 converts to a huge unsigned value
unsigned u = 0;  u--;                          // wraps to UINT_MAX (well-defined for unsigned)
int k = INT_MAX;  k++;                         // signed overflow: UNDEFINED
printf("%d", 5/2);  printf("%f", 5/2);         // 2, then undefined (int passed for %f); use 5/2.0 ->
    2.5
printf("%d", (int)'A');                        // 65
char c = 200;  printf("%d", c);                // implementation-defined: usually -56 (signed char)
int a = 3;  printf("%d", a == 3 ? 10 : 20);     // 10
printf("%d", !!5);                             // 1
if (x = 0) {...}                               // assignment, not comparison: never true; always warns
    with -Wall
```

`printf("%s", 0)` prints `(null)` on glibc but is UB. `i = i++` is UB before C++17 and still dubious in C.

Q8. Pre-processor, `#define` vs `const`, macro pitfalls.

`#define` is textual substitution before compilation: no type, no scope.

```
#define SQUARE(x) x*x
SQUARE(2+3)            // expands to 2+3*2+3 = 11, not 25
#define SQ(x) ((x)*(x))
SQ(i++)                // i++ evaluated twice: UB
```

Prefer `const`/`constexpr`/`inline` functions. Include guards (`#ifndef X_H`, `#pragma once`) prevent double inclusion. `#include <>` searches system paths; `""` searches the local directory first.

Q9. Bit manipulation essentials.

```
x & (x-1)              // clears lowest set bit; x & (x-1) == 0 -> power of two (x > 0)
x & -x                 // isolates lowest set bit
(x >> k) & 1           // test bit k        x |= (1u<<k)   set        x &= ~(1u<<k)  clear     x ^=
    (1u<<k)  toggle
__builtin_popcount(x)  // number of set bits (GCC/Clang)
a ^= b; b ^= a; a ^= b;   // swap without temp (fails if a and b alias the same variable)
```

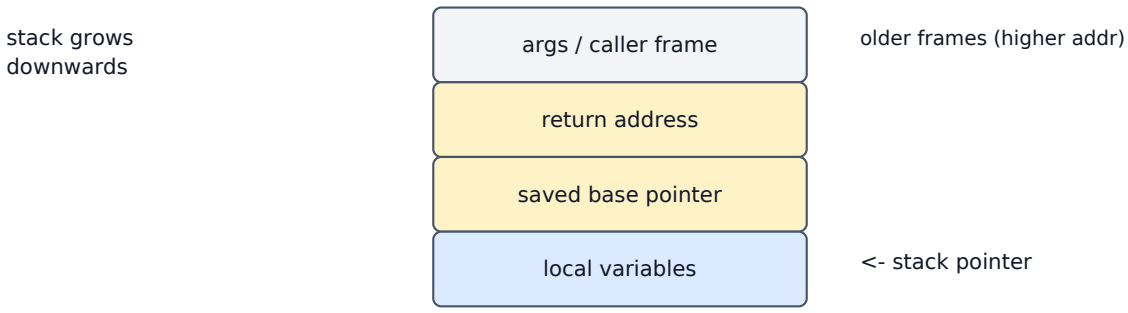
Right shift of a negative signed number is implementation-defined (usually arithmetic). `1 << 31` on a 32-bit `int` is UB; use `1u << 31`.

Q10. static, extern, volatile, register, `const` in C. Storage classes.

`auto` (default for locals), `register` (hint, can't take address), `static` (local: persists across calls, global: file-private), `extern` (declared elsewhere). Linkage: external (visible to other files) vs internal (`static`) vs none (locals). A global defined in a header and included in two `.c` files gives duplicate symbol errors: declare with `extern` in the header and define once in a `.c`.

Q11. How does a function call work on the stack? What is a stack overflow?

On a call, the caller pushes arguments (or uses registers), the **return address** is pushed, the callee sets up a frame (saves the old base pointer, reserves space for locals). On return, the frame is popped.



Infinite or very deep recursion exhausts the stack (stack overflow). Writing past a local buffer can overwrite the return address (the classic buffer-overflow exploit; mitigated by stack canaries, ASLR, NX).

Q12. Dry run: what does this recursion print?

```
void f(int n){ if(n==0) return;  printf("%d ", n);  f(n-1);  printf("%d ", n); }
f(3);
```

Calls: `f(3)` prints 3, `f(2)` prints 2, `f(1)` prints 1, `f(0)` returns; then unwinding prints 1, 2, 3.

Output: `3 2 1 1 2 3`

Edge cases and gotchas

- `sizeof` is an operator evaluated at compile time (except VLAs); `sizeof(int[10])` is 40.
- Array out-of-bounds may "work" silently; that is the dangerous case.
- `char` signedness is implementation-defined: use `unsigned char` for bytes.
- `malloc(0)` may return NULL or a unique pointer; both are legal.
- Returning a pointer to a `static` local is legal (one shared copy, not thread-safe).
- Stack variables are uninitialised; globals/statics are zero-initialised.
- `gets()` is removed from the standard (unbounded read); use `fgets`.
- `int main` returning without a value in C99+/C++ implicitly returns 0.
- Uninitialised pointer vs `NULL` pointer: both crash on dereference, but only NULL is checkable.
- Comparing floats with `==` fails due to representation error (`0.1 + 0.2 != 0.3`).

Rapid-fire round

Question	Answer
Stack vs heap	Automatic, fast, limited vs manual, flexible, can fragment
`malloc` vs `calloc`	Uninitialised vs zero-filled
Dangling pointer	Points to freed/out-of-scope memory
Memory leak	Allocated but never freed and unreachable
`strlen` vs `sizeof`	Chars before `\0` at run time vs bytes of the object at compile time
Segmentation fault	Access to memory the process may not touch (null, freed, out of bounds)
Little vs big endian	LSB at lowest address vs MSB at lowest address
`volatile`	Don't optimise away reads/writes; not atomic
Why `a[i] == i[a]`	Both mean `*(a+i)`
Header guard purpose	Avoid multiple inclusion

Part B — Operating Systems

Order: basics and processes, threads, scheduling (with Gantt dry runs), synchronisation, deadlock, memory management (with page-replacement dry runs), then file systems and disk. Numerical problems are worked step by step.

OS Basics, Processes and Threads

Most asked: kernel vs user mode, system calls, process vs thread, process states, context switch, fork() output puzzles, IPC, zombie/orphan.

Q1. What is an operating system and what are its main functions?

Software that manages hardware and provides services to programs: **process management** (create, schedule), **memory management** (allocation, virtual memory, protection), **file system**, **device/I-O management**, **security and protection**, and a user interface (shell/GUI). Goals: convenience, efficient resource use, isolation.

Q2. Kernel mode vs user mode; what is a system call?

The CPU has a mode bit. **User mode** cannot execute privileged instructions or touch hardware; **kernel mode** can. A **system call** is the controlled entry from user to kernel (`read`, `write`, `fork`, `open`, `mmap`): the program executes a trap instruction, the CPU switches to kernel mode and jumps to the handler, the kernel does the work and returns.



Interrupts (from hardware), exceptions/traps (page fault, divide by zero) and system calls all enter the kernel.

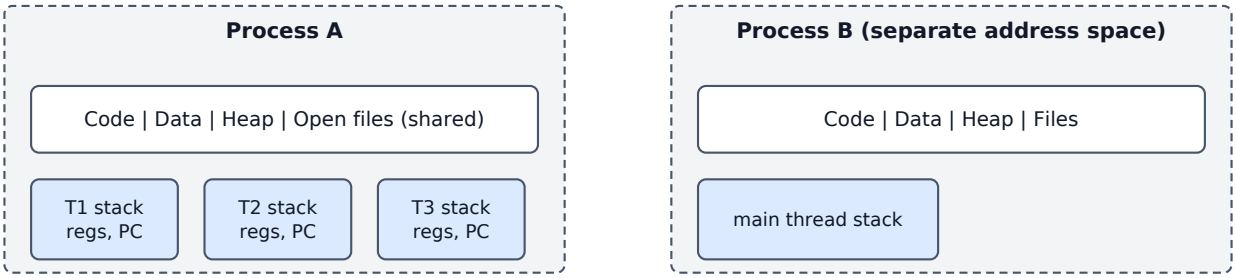
Q3. Monolithic kernel vs microkernel. What is a hypervisor?

Monolithic (Linux): most services in kernel space, fast but one bug can crash everything. Microkernel (Minix, QNX): minimal kernel (IPC, scheduling, memory), other services in user space; safer and modular but more message-passing overhead. Hybrid (Windows NT, macOS) mixes both. A **hypervisor** runs multiple virtual machines (Type 1 on hardware, Type 2 on a host OS).

Q4. Process vs program vs thread.

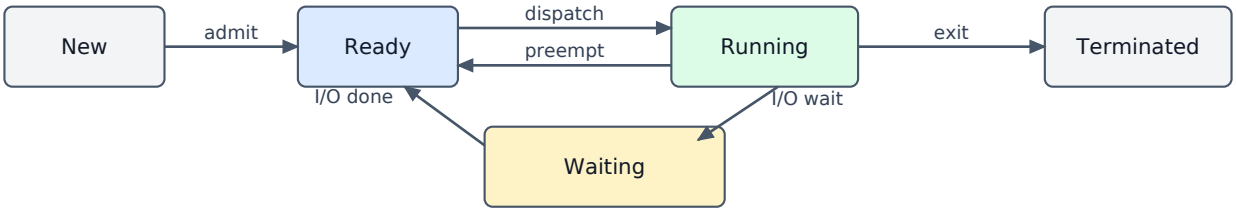
A program is a passive file on disk; a **process** is a running instance with its own address space (code, data, heap, stack), open files and a **PCB**. A **thread** is a unit of execution inside a process: threads share code, data, heap and open files but each has its own **stack, registers and program counter**.

	Process	Thread
Address space	Separate	Shared within process
Creation / context switch	Heavy	Light
Communication	IPC needed (pipes, sockets, shared memory)	Direct via shared memory (needs sync)
Failure	Isolated	One thread crash can kill the process



Q5. What is a PCB and what are the process states?

The **Process Control Block** stores PID, state, program counter, CPU registers, scheduling info, memory maps, open files, accounting. States: **New**, **Ready**, **Running**, **Waiting/Blocked** (for I/O or event), **Terminated**.



Q6. What is a context switch and why is it expensive?

Saving the state (registers, PC, stack pointer) of the running process/thread into its PCB/TCB and loading another's. Overhead: no useful work during the switch, **TLB flush** (for process switches, since the address space changes), cache and branch-predictor pollution. Thread switches within one process are cheaper (no address-space change).

Q7. Explain `fork()` and predict outputs (dry runs).

`fork()` creates a child that is a copy of the caller (copy-on-write). It returns **0 in the child**, the child's PID in the parent, and -1 on failure. Execution continues from the line after `fork()` in both.

```
printf("A "); fork(); printf("B ");
```

Output: `A B B` (A printed once, since it ran before the fork; B printed by both). Caveat: if stdout is a pipe/file, `A` can sit in the buffer and be duplicated by the fork, giving `A B A B`. Use `fflush` or `\n` on terminals.

```
fork(); fork(); fork();  printf("x ");     // 2^3 = 8 processes, so "x" printed 8 times
```

Number of processes after `n` unconditional forks is `2^n`; the number of *new* children is `2^n - 1`.

```
int x = 10;
if (fork() == 0) { x += 5; printf("child %d\n", x); }     // child 15
else { wait(NULL); printf("parent %d\n", x); }            // parent 10 (separate address spaces)
```

`exec()` replaces the process image with a new program (PID unchanged). Shell pattern: `fork` then `exec` in child, `wait` in parent.

Q8. Zombie vs orphan process.

Zombie: a child that has terminated but whose exit status has not been read by the parent with `wait()`, so its PCB entry remains (shows `Z` in `ps`). Cannot be killed (it is already dead); fix by having the parent `wait` or by killing the parent. **Orphan**: parent exits first; the child is adopted by `init`/`systemd` (PID 1), which reaps it. **Daemon**: a background service process, usually detached from a terminal.

Q9. Inter-process communication mechanisms.

Pipes (unidirectional, related processes), named pipes/FIFOs, message queues, **shared memory** (fastest, needs synchronisation), semaphores, sockets (also across machines), signals (`SIGKILL`, `SIGTERM`, `SIGINT`; `SIGKILL` and `SIGSTOP` cannot be caught), memory-mapped files.

Q10. User-level vs kernel-level threads; threading models.

User-level threads are managed by a library, cheap to switch, but one blocking system call blocks all and they cannot use multiple cores by themselves. Kernel-level threads are scheduled by the OS: true parallelism, higher overhead. Models: many-to-one, one-to-one (Linux, Windows), many-to-many. Modern systems use one-to-one.

Q11. Multithreading vs multiprocessing; concurrency vs parallelism.

Concurrency: dealing with many things at once (interleaving, possibly on one core). **Parallelism**: doing many things at the same time (multiple cores). Use threads for shared-state, I/O-heavy work; use processes for isolation or to dodge a runtime lock (Python's GIL). Thread pools bound resource usage and avoid creation cost.

Edge cases and gotchas

- Child gets a *copy* of memory (copy-on-write): changing a variable in the child does not affect the parent.
- Buffered `printf` before `fork` can be printed twice.
- `fork()` in a multithreaded program only duplicates the calling thread: locks held by other threads stay locked in the child.
- A zombie does not consume memory beyond its PCB, but too many exhaust the PID table.
- `wait()` returns when *any* child terminates; `waitpid` targets a specific one.
- Thread stack size is limited; deep recursion in a thread can overflow sooner.
- `exec` does not return on success.
- Signal handlers should only call async-signal-safe functions.
- Thread-local storage gives each thread its own copy of a "global".

Rapid-fire round

Question	Answer
PCB	Kernel record describing a process
What does `fork` return in the child	0
Number of processes after 4 forks	16 total (15 new)
Process states	New, Ready, Running, Waiting, Terminated
What do threads share	Code, data, heap, files; not stack/registers
Cheapest IPC	Shared memory (needs sync)
Which signals cannot be caught	SIGKILL, SIGSTOP
Who reaps orphans	init / systemd
Trap vs interrupt	Software-generated (exception/syscall) vs hardware-generated

CPU Scheduling

Most asked: FCFS/SJF/SRTF/Round Robin/Priority numericals with Gantt charts, average waiting and turnaround time, convoy effect, starvation and aging, MLFQ.

Q1. Define the scheduling terms and formulas.

Arrival time (AT), **burst time (BT)**, **completion time (CT)**. **Turnaround time (TAT) = CT - AT**. **Waiting time (WT) = TAT - BT**. **Response time = first CPU time - AT**. Also CPU utilisation, throughput. **Preemptive** schedulers can take the CPU away; non-preemptive wait until the process blocks or ends.

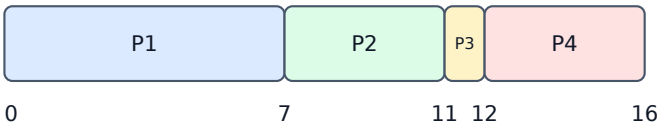
Q2. Compare FCFS, SJF, SRTF, Round Robin, Priority.

Algorithm	Type	Pros	Cons
FCFS	Non-preemptive	Simple, fair order	**Convoy effect**: short jobs wait behind a long one
SJF	Non-preemptive	Minimum average WT (optimal)	Needs burst estimate; **starvation** of long jobs
SRTF	Preemptive SJF	Optimal average WT	More context switches, starvation
Round Robin	Preemptive, quantum q	Good response time, no starvation	Large q becomes FCFS; tiny q wastes time switching
Priority	Either	Reflects importance	Starvation; fix with **aging**

Q3. Worked example: FCFS vs SJF vs SRTF.

Processes: P1 (AT 0, BT 7), P2 (AT 2, BT 4), P3 (AT 4, BT 1), P4 (AT 5, BT 4).

FCFS order P1, P2, P3, P4:



CT: P1=7, P2=11, P3=12, P4=16. TAT = CT - AT: 7, 9, 8, 11 (avg 8.75). WT = TAT - BT: 0, 5, 7, 7 (avg 4.75). Note P3 (burst 1) waited 7: the convoy effect.

SJF (non-preemptive): at t=0 only P1 is there, so it runs 0-7. At t=7, ready: P2 (4), P3 (1), P4 (4). Shortest is P3 (7-8), then tie P2 and P4 (4 each), pick earlier arrival P2 (8-12), then P4 (12-16).

CT: P1=7, P3=8, P2=12, P4=16. TAT: 7, 10, 4, 11 (avg 8.0). WT: 0, 6, 3, 7 (avg 4.0).

SRTF (preemptive SJF): t=0 P1 runs. t=2 P2 arrives (4) vs P1 remaining 5, so P2 preempts, runs 2-4. t=4 P3 arrives (1) vs P2 remaining 2, so P3 runs 4-5. t=5: P4 (4) arrives; ready are P2 (rem 2), P1 (rem 5), P4 (4): P2 runs 5-7. Then P4 (7-11), then P1 (11-16).

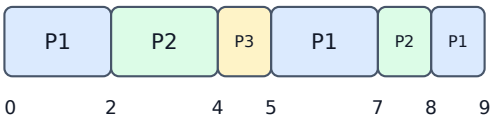
CT: P3=5, P2=7, P4=11, P1=16. TAT: P1 16, P2 5, P3 1, P4 6 (avg 7.0). WT: P1 9, P2 1, P3 0, P4 2 (avg 3.0).

Order of average WT: SRTF 3.0 < SJF 4.0 < FCFS 4.75.

Q4. Worked example: Round Robin (quantum 2).

P1 (AT 0, BT 5), P2 (AT 1, BT 3), P3 (AT 2, BT 1). Ready queue evolves; rule: a process that arrives while another runs is queued *before* the preempted one is re-queued.

- t=0-2: P1 runs (rem 3). P2 arrived at 1, P3 at 2. Queue: P2, P3, P1.
- t=2-4: P2 runs (rem 1). Queue: P3, P1, P2.
- t=4-5: P3 runs (needs 1), done at 5.
- t=5-7: P1 runs (rem 1). Queue: P2, P1.
- t=7-8: P2 runs, done at 8.
- t=8-9: P1 runs, done at 9.



CT: P1=9, P2=8, P3=5. TAT: 9, 7, 3 (avg 6.33). WT = TAT - BT: 4, 4, 2 (avg 3.33). Response time: P1=0, P2=1, P3=2.

Q5. Starvation, aging, convoy effect, and MLFQ.

Starvation: a process waits indefinitely (low priority, long job under SJF). **Aging** gradually raises the priority of waiting processes. **Convoy effect**: many short processes queue behind one CPU-bound process (FCFS). **Multilevel Feedback Queue**: several queues with different quanta/priorities; new jobs enter the top queue, a job that uses its whole quantum is demoted (CPU-bound sinks), a job that blocks early stays high (interactive); periodic boost prevents starvation. Real systems (Linux CFS) approximate fair sharing with virtual runtime and a red-black tree.

Q6. Preemptive vs non-preemptive; short-term vs medium vs long-term scheduler; dispatcher.

Long-term scheduler (job scheduler) decides which jobs are admitted to memory; short-term (CPU scheduler) picks the next ready process, frequently; medium-term swaps processes out/in. The **dispatcher** performs the actual switch (context switch, mode switch, jump); its delay is **dispatch latency**.

Edge cases and gotchas

- With Round Robin, if all processes arrive at 0, the tie-breaking and queue order affect the Gantt chart; state your assumption.
- SJF is optimal for average waiting time only if burst times are known (they are estimated by exponential averaging).
- RR with an extremely large quantum becomes FCFS; with a very small quantum the overhead dominates.
- Idle time: if no process has arrived, the CPU idles; completion times jump.
- Priority inversion: a low-priority process holds a lock needed by a high-priority one while a medium one runs; fixed by **priority inheritance** (Mars Pathfinder bug).
- Real-time scheduling: Rate Monotonic (static priority by period), Earliest Deadline First (dynamic).

Rapid-fire round

Question	Answer
TAT formula	CT - AT
WT formula	TAT - BT
Optimal average WT	SJF / SRTF (SRTF for preemptive)
Algorithm with convoy effect	FCFS
Fix for starvation	Aging
RR needs a	Time quantum
Which is preemptive among FCFS/SJF/SRTF/RR	SRTF and RR
Dispatch latency	Time the dispatcher takes to stop one process and start another

Synchronisation and Deadlock

> Most asked: race conditions, critical section requirements, mutex vs semaphore, producer-consumer, readers-writers, dining philosophers, the four deadlock conditions, Banker's algorithm.

Q1. What is a race condition? Dry run `counter++` with two threads.

A race condition is when the result depends on the unpredictable interleaving of threads accessing shared data. `counter++` is **not atomic**; it compiles to three steps: load, add, store.

```
Initial counter = 5, threads T1 and T2 each run counter++
T1: load  r1 = 5
T2: load  r2 = 5          <- T2 reads before T1 stores
T1: r1 = 6, store counter = 6
T2: r2 = 6, store counter = 6   <- one increment lost; expected 7
```

Fix: make the sequence atomic with a mutex, an atomic type (`std::atomic<int>`, `fetch_add`), or avoid sharing.

Q2. What is the critical section problem? What must a solution guarantee?

The critical section is code that accesses shared data. A solution needs: **Mutual exclusion** (one at a time), **Progress** (if nobody is inside, a waiting process must be able to enter; decision cannot be postponed forever), **Bounded waiting** (no starvation: a limit on how many others can enter before a waiting process). Peterson's algorithm solves it for two processes in software:

```
flag[i] = true;  turn = j;              // "I want in, but you go first"
while (flag[j] && turn == j) ;          // wait
   /* critical section */
flag[i] = false;
```

On modern CPUs with reordering it needs memory barriers; real systems use hardware atomics (`test-and-set`, `compare-and-swap`).

Q3. Mutex vs semaphore vs monitor vs spinlock.

	Mutex	Semaphore	Spinlock
Purpose	Mutual exclusion (lock/unlock)	Signalling and resource counting	Short mutual exclusion
Value	Locked/unlocked	Integer >= 0	Locked/unlocked
Ownership	Only the locker may unlock	Any thread may `signal`	Owner
Waiting	Sleeps (blocks)	Sleeps	**Busy-waits** (burns CPU)

Semaphore ops: `wait(S)`: `S--`, block if negative (P); `signal(S)`: `S++`, wake one (V). Binary semaphore (0/1) resembles a mutex but without ownership. Spinlocks suit very short critical sections on multicore systems; blocking locks suit long ones. A **monitor** is a language-level construct bundling shared data, procedures and condition variables, where only one thread is active inside at a time (Java `synchronized`, C# `lock`).

Q4. Condition variables: why the `while` loop?

`cv.wait(lock)` atomically releases the lock and sleeps; on wake-up it re-acquires the lock. Always re-check the condition in a **loop**, because of spurious wake-ups and because another thread may have consumed the state before you ran.

```
std::unique_lock<std::mutex> lk(m);
cv.wait(lk, []{ return !queue.empty(); });   // predicate form loops internally
```

Q5. Producer-consumer (bounded buffer) with semaphores.

```
semaphore empty = N;    // free slots
semaphore full  = 0;    // filled slots
semaphore mutex = 1;    // protects the buffer

producer:  wait(empty);  wait(mutex);  put(item);  signal(mutex);  signal(full);
consumer:  wait(full);   wait(mutex);  get(item);  signal(mutex);  signal(empty);
```

Order matters: if the producer did `wait(mutex)` before `wait(empty)` and the buffer is full, it sleeps holding the mutex and the consumer can never enter: **deadlock**. Dry run with N=1: producer `wait(empty)` (1 to 0), puts, `signal(full)` (0 to 1). A second producer blocks at `wait(empty)` until the consumer's `signal(empty)`.

Q6. Readers-writers problem.

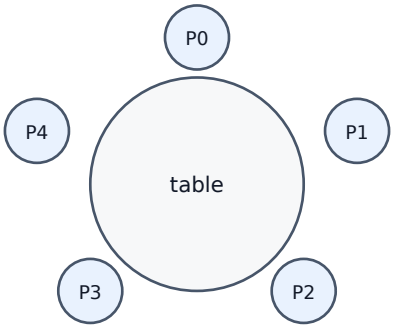
Many readers may read together; a writer needs exclusive access. First-readers-writers: readers have priority (writers may starve).

```
reader:  wait(rmutex); if(++readcount==1) wait(wrt); signal(rmutex);
         read();
         wait(rmutex); if(--readcount==0) signal(wrt); signal(rmutex);
writer:  wait(wrt);  write();  signal(wrt);
```

The first reader locks writers out, the last reader lets them in. In practice use a reader-writer lock (`std::shared_mutex`).

Q7. Dining philosophers: the problem and solutions.

Five philosophers, five forks; each needs both neighbouring forks. If everyone picks up the left fork first, all hold one and wait forever: **deadlock**.



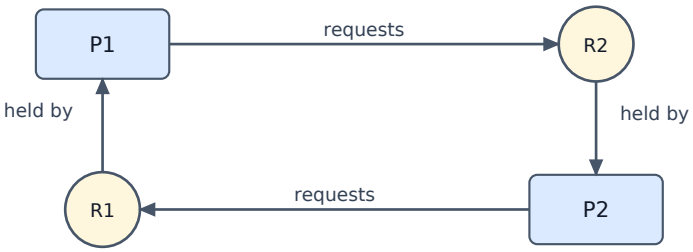
fork between each pair

Solutions: allow at most 4 philosophers to sit; pick up both forks atomically (inside a mutex); **resource ordering** (always take the lower-numbered fork first, which breaks the circular wait); asymmetric (odd philosophers take left first, even take right first).

Q8. What is deadlock? State the four necessary (Coffman) conditions.

Deadlock: a set of processes each waiting for a resource held by another in the set. All four must hold:

1. **Mutual exclusion**: resource is non-shareable.
2. **Hold and wait**: holds one resource while waiting for others.
3. **No preemption**: resources cannot be forcibly taken.
4. **Circular wait**: a cycle of waiting processes.



P1 holds R1, wants R2

P2 holds R2, wants R1

cycle = circular wait

Classic code: thread A locks `m1` then `m2`; thread B locks `m2` then `m1`.

```
A: lock(m1); lock(m2);  ...        B: lock(m2); lock(m1);  ...     // may deadlock
```

Fix: a global lock order, `std::scoped_lock(m1, m2)` (acquires both without deadlock), or `try_lock` with back-off.

Q9. Deadlock prevention, avoidance, detection and recovery.

- **Prevention**: negate one condition: acquire all resources at once (hold-and-wait), allow preemption, order resources (circular wait). Costly or restrictive.
- **Avoidance**: grant a request only if the system stays in a **safe state** (Banker's algorithm). Needs maximum demand in advance.
- **Detection and recovery**: allow deadlock, detect via cycle in the resource-allocation graph or an algorithm like Banker's; recover by killing processes or preempting resources.
- **Ignore** (ostrich algorithm): used by general-purpose OSes; deadlocks are rare and the cost of prevention is high.

A cycle in a resource graph means deadlock **only if each resource has a single instance**; with multiple instances a cycle is necessary but not sufficient.

Q10. Banker's algorithm: worked safety check.

Resources A, B, C. Available = (3, 3, 2).

Process	Allocation	Max	Need = Max - Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Safety algorithm: find a process whose Need <= Work, assume it finishes and returns its allocation.

- Work (3,3,2): P1 need (1,2,2) fits, Work becomes (5,3,2).
- P3 need (0,1,1) fits, Work (7,4,3).
- P4 need (4,3,1) fits, Work (7,4,5).
- P0 need (7,4,3) fits, Work (7,5,5).
- P2 need (6,0,0) fits, Work (10,5,7).

Safe sequence: **P1, P3, P4, P0, P2**. The state is safe. A *safe* state guarantees no deadlock; an *unsafe* state may (not must) lead to one.

Q11. Livelock, starvation, priority inversion.

Livelock: processes keep changing state in response to each other but make no progress (two people stepping aside in a corridor). **Starvation**: a process never gets the resource. **Priority inversion**: low-priority holder blocks a high-priority waiter while a medium-priority task runs; solved by priority inheritance.

Edge cases and gotchas

- Mutex unlock by a thread that does not own it is undefined; semaphores allow it.
- A recursive lock is needed if the same thread re-locks (`std::recursive_mutex`), otherwise it self-deadlocks.
- Spinlock on a single-core machine is useless (the lock holder cannot run while you spin).
- Lost wake-up: signalling a condition variable before anyone waits; always pair with a state variable checked under the lock.
- Double-checked locking is broken without atomics/memory barriers.
- `volatile` does not provide atomicity or ordering between threads.
- Lock granularity: coarse locks are simple but limit parallelism; fine-grained locks risk deadlock.

► Lock-free structures use CAS but face the ABA problem.

Rapid-fire round

Question	Answer
Four deadlock conditions	Mutual exclusion, hold and wait, no preemption, circular wait
Semaphore ops	wait/P (decrement), signal/V (increment)
Mutex ownership	Only the owner unlocks
Safe state means	A safe sequence exists, so no deadlock
Cycle in RAG, single instance	Deadlock
Easiest condition to break	Circular wait via resource ordering
Spinlock use	Very short critical sections on multicore
Atomic operation	Indivisible, cannot be interleaved

Memory Management and Virtual Memory

Most asked: paging vs segmentation, address translation, page table and TLB, effective access time numericals, page faults, replacement algorithms with dry runs, Belady's anomaly, thrashing, fragmentation.

Q1. Logical vs physical address. What does the MMU do?

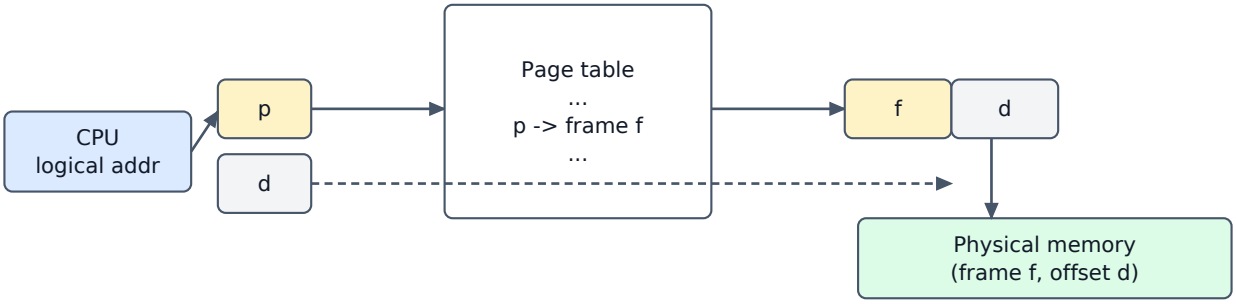
A program uses **logical (virtual) addresses**; the **MMU** (hardware) translates them to **physical addresses** at run time. This gives relocation, isolation and the illusion of a large contiguous memory.

Q2. Contiguous allocation: fragmentation and placement strategies.

External fragmentation: free memory is split into holes too small to use. **Internal fragmentation**: allocated block is larger than needed (wasted inside). Placement: **first fit** (first hole big enough, fast), **best fit** (smallest adequate hole, leaves tiny slivers), **worst fit** (largest hole). Fix external fragmentation with compaction or paging.

Q3. Paging: how does address translation work? (numerical)

Physical memory is divided into **frames**, logical memory into **pages** of the same size. A logical address = (page number p, offset d). The **page table** maps p to frame f; physical address = f x page size + d.



Example: 32-bit logical address, page size 4 KB (2^12). Offset = 12 bits, page number = 20 bits, so the page table has 2^20 = 1,048,576 entries (4 MB at 4 bytes each, per process). If page 5 maps to frame 9, logical address 5 x 4096 + 100 = 20580 translates to 9 x 4096 + 100 = 36964.

Paging removes external fragmentation but has some internal fragmentation (last page). Large tables are handled with **multi-level page tables**, **hashed** or **inverted** page tables.

Q4. What is the TLB? Compute the effective access time.

The **Translation Lookaside Buffer** is a small, fast cache of recent page-to-frame mappings. A TLB hit avoids the extra memory access to the page table.

EAT with TLB lookup 10 ns, memory access 100 ns, TLB hit ratio 90%:

- Hit: 10 + 100 = 110 ns. Miss: 10 + 100 (page table) + 100 (data) = 210 ns.
- EAT = 0.9 x 110 + 0.1 x 210 = 99 + 21 = **120 ns**.

On a context switch to another process the TLB is flushed (or tagged with address-space IDs, ASID).

Q5. Paging vs segmentation.

Paging: fixed-size pages, invisible to the programmer, no external fragmentation. Segmentation: variable-size logical units (code, stack, heap) with base and limit, matches the program's view and eases sharing/protection, but has external fragmentation. Many systems combine them (segmented paging).

Q6. What is virtual memory and demand paging?

Virtual memory lets a process run when only part of it is in RAM; pages are loaded **on demand** from disk. Benefits: programs larger than RAM, more processes in memory, simple sharing (shared libraries), copy-on-write. Each page-table entry has a **valid/present bit**, dirty bit, reference bit, protection bits.

Q7. Walk through a page fault.

1. CPU references a page whose present bit is 0, causing a trap to the OS.
2. OS checks the reference is legal (else segmentation fault).
3. Find a free frame (or choose a victim with a replacement algorithm; write it back if dirty).
4. Read the page from disk into the frame (the process blocks; another runs).
5. Update the page table and TLB, set the present bit.
6. Restart the faulting instruction.

Page-fault service is ~milliseconds versus ~100 ns memory access, so even a tiny fault rate hurts: with fault rate p, EAT = (1 - p) x 100 ns + p x 8 ms; p = 1/1000 gives about 8.1 microseconds, 80 times slower.

Q8. Page replacement dry runs: FIFO, LRU, Optimal.

Reference string: `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1`, **3 frames**.

FIFO (evict the oldest loaded page). Faults at: 7,0,1 (3 cold), 2 (evict 7), 0 hit, 3 (evict 0), 0 (evict 1), 4 (evict 2), 2 (evict 3), 3 (evict 0), 0 (evict 4), 3 hit, 2 hit, 1 (evict 3), 2 hit, 0 hit, 1 hit, 7 (evict 2), 0 hit, 1 hit. Total **15 faults**.

LRU (evict the least recently used): total **12 faults**.

Optimal (evict the page used farthest in the future; not implementable, a benchmark): total **9 faults**.

Check a few LRU steps: after `7 0 1` frames {7,0,1}; page 2 evicts 7 (least recent); 0 hit; 3 evicts 1 (LRU, since 0 was just used); 0 hit; 4 evicts 2; 2 evicts 3; 3 evicts 0; 0 evicts 4; ...

Belady's anomaly: with FIFO, more frames can give *more* faults. String `1 2 3 4 1 2 5 1 2 3 4 5`: 3 frames gives **9** faults, 4 frames gives **10**. LRU and Optimal are stack algorithms and never show it.

Other policies: **Second chance / Clock** (FIFO with a reference bit), **LFU**, **NRU**.

Q9. Thrashing and the working set.

Thrashing: the system spends more time paging than executing because processes lack enough frames; CPU utilisation falls, and a naive scheduler admits more processes, making it worse. The **working set** is the set of pages a process actively uses in a recent window; keep it in memory, or reduce the degree of multiprogramming. Page-fault frequency (PFF) control adds or removes frames based on the fault rate.

Q10. Copy-on-write, memory-mapped files, swapping.

Copy-on-write: after `fork`, parent and child share pages marked read-only; a write triggers a page fault and the kernel copies just that page. **mmap** maps a file into the address space so file access becomes memory access. **Swapping** moves whole processes (or pages) between RAM and disk (swap space).

Q11. How do `malloc` and the heap work? What is the OOM killer?

`malloc` is a user-space allocator that manages heap blocks (free lists, size classes) and asks the kernel for more memory via `brk`/`sbrk` or `mmap` (large requests). Linux can **overcommit**; when memory is exhausted, the **OOM killer** terminates a process chosen by a badness score.

Edge cases and gotchas

- Page size trade-off: small pages reduce internal fragmentation but enlarge page tables and TLB pressure; large pages (huge pages) do the opposite.
- Two-level paging cuts memory used by sparse address spaces: only allocated second-level tables exist.
- A cold start hits many page faults (demand paging); `mlock` pins pages.
- A dirty page must be written back before eviction; clean pages can be dropped.
- Segmentation fault vs page fault: a page fault can be valid (page just not loaded); a segfault is an illegal access.
- Thrashing is not fixed by a faster CPU.
- Stack algorithms (LRU) guarantee the set of pages with n frames is a subset of n+1 frames.

Rapid-fire round

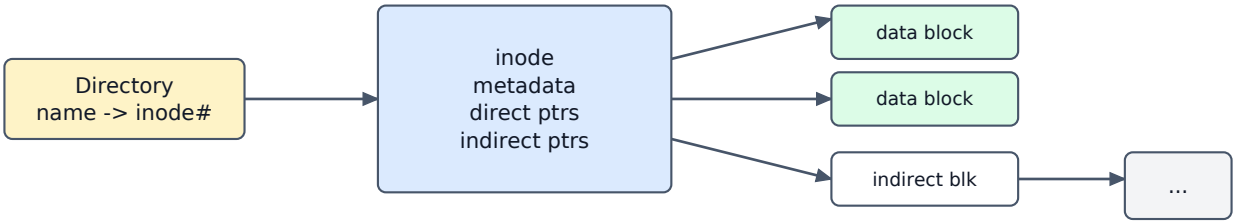
Question	Answer
MMU	Hardware translating virtual to physical addresses
Page size 4 KB offset bits	12
TLB	Cache of page-table entries
Belady's anomaly algorithm	FIFO
Best page replacement (theory)	Optimal
Thrashing cause	Too little memory per process, excess paging
Internal vs external fragmentation	Waste inside a block vs between blocks
Page fault handled by	OS kernel (trap)
Inverted page table	One entry per physical frame, not per page

File Systems, Disk and I/O

Most asked: file allocation methods, inodes, disk scheduling (SSTF/SCAN/C-SCAN numericals), hard vs soft link, RAID, buffering and DMA.

Q1. What does a file system provide? What is an inode?

Files, directories, naming, access control, and mapping of file blocks to disk. In Unix, an **inode** stores metadata (owner, permissions, size, timestamps, link count) and pointers to data blocks (direct, single, double, triple indirect) but **not the file name** (names live in directory entries that map name to inode number).



Q2. Hard link vs soft (symbolic) link.

	Hard link	Soft link
Points to	Same inode	A path (separate file)
Survives deletion of original	Yes (data lives while link count > 0)	No (dangling)
Across file systems	No	Yes
To directories	Generally not	Yes

Deleting a file removes a name and decrements the inode's link count; data is freed when it reaches 0 and no process has it open.

Q3. File allocation methods.

Method	Idea	Pros	Cons
Contiguous	File in consecutive blocks	Fast sequential and random access	External fragmentation, hard to grow
Linked	Each block points to next	No external fragmentation	Slow random access, pointer loss risk (FAT puts links in a table)
Indexed	Index block lists all blocks	Good random access, no fragmentation	Index overhead (inode uses multi-level)

Q4. Disk scheduling: worked example.

Disk with cylinders 0-199, head at **53**, queue: `98, 183, 37, 122, 14, 124, 65, 67`.

FCFS: 53 to 98 (45), 183 (85), 37 (146), 122 (85), 14 (108), 124 (110), 65 (59), 67 (2): total **640**.

SSTF (nearest first): 53 to 65 (12), 67 (2), 37 (30), 14 (23), 98 (84), 122 (24), 124 (2), 183 (59): total **236**. Can starve far requests.

SCAN (elevator), head moving toward 0 first: 53 to 37 (16), 14 (23), 0 (14), then up to 65 (65), 67 (2), 98 (31), 122 (24), 124 (2), 183 (59): total 53 + 183 = **236** (53 down to 0, then 0 up to 183).

C-SCAN: services in one direction, then jumps back to the start without servicing on the return, giving more uniform wait times. **LOOK/C-LOOK** go only as far as the last request instead of the disk end.

For SSDs seek time is irrelevant, so schedulers matter less (noop/deadline).

Q5. Buffering, caching, spooling, DMA, polling vs interrupts.

Buffering smooths speed mismatches; **caching** keeps copies of hot data; **spooling** queues output for a device (printer). **DMA** lets a device transfer data to memory without the CPU copying each byte; the CPU only starts the transfer and takes one interrupt at the end. **Polling** repeatedly checks device status (wastes CPU); **interrupts** signal when ready.

Q6. RAID levels.

RAID 0: striping, speed, no redundancy. RAID 1: mirroring, survives one disk loss. RAID 5: striping with distributed parity, survives one disk failure (n-1 usable). RAID 6: double parity (survives two). RAID 10: mirrored stripes (fast and redundant, half capacity). RAID is not a backup.

Q7. Journaling, `fsync`, and what happens on `open()`/`read()`?

A journaling file system (ext4, NTFS) logs intended metadata changes before applying them so crash recovery is fast and consistent. `write()` normally goes to the page cache; `fsync` forces data to disk. `open` resolves the path through directories to an inode, checks permissions and returns a **file descriptor** (index into the per-process table, which points to a system-wide open-file table entry holding the offset). FDs 0, 1, 2 are stdin, stdout, stderr.

Edge cases and gotchas

- Deleting an open file: name disappears but data persists until the last descriptor closes.
- Inode exhaustion: disk can have free space but no free inodes (many tiny files).
- `rm` removes the name, not necessarily the data.
- SCAN average wait is worse for requests just behind the head.
- Writes are buffered: power loss before `fsync` can lose "written" data.
- Fork duplicates the descriptor table; both processes share the same file offset.

Rapid-fire round

Question	Answer
Does an inode store the file name	No
Best disk scheduling for fairness	C-SCAN / LOOK variants
SSTF problem	Starvation
RAID that survives 1 disk failure with parity	RAID 5
File descriptor for stderr	2
DMA purpose	Transfer data without CPU copying
Hard link limit	Same file system only
Journaling benefit	Fast, consistent crash recovery

Part C — Computer Networks

Order: layered models and the data-link layer, IP addressing with subnetting numericals, routing, transport (TCP/UDP with handshakes and congestion control), application protocols (DNS, HTTP, TLS), and the "what happens when you type a URL" walkthrough.

Network Models, Link Layer and Devices

> Most asked: OSI vs TCP/IP, encapsulation, switch vs router vs hub, MAC vs IP, ARP, Ethernet, error detection, CSMA/CD, flow control (stop-and-wait, sliding window).

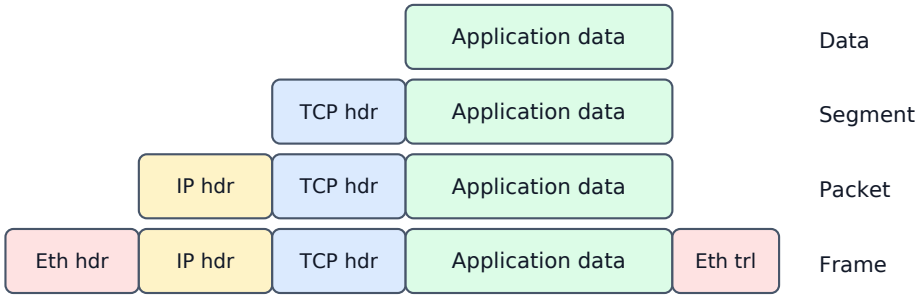
Q1. Explain the OSI model and the TCP/IP model. Name a protocol and a PDU per layer.

OSI layer	TCP/IP layer	Examples	PDU
7 Application	Application	HTTP, DNS, SMTP, FTP, SSH	Data/message
6 Presentation		TLS, encoding, compression	
5 Session		Sessions, RPC	
4 Transport	Transport	TCP, UDP	Segment / datagram
3 Network	Internet	IP, ICMP, routing	Packet
2 Data link	Network access	Ethernet, Wi-Fi, ARP	Frame
1 Physical		Cables, signals, bits	Bits

Mnemonic (top to bottom): **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing. TCP/IP is what is actually implemented; OSI is the teaching reference.

Q2. What is encapsulation?

Each layer adds its own header (the link layer also a trailer) around the data from the layer above; the receiver strips them in reverse.



Q3. Hub vs switch vs router vs bridge vs gateway.

Device	Layer	Forwards by	Collision / broadcast domains
Hub	1	Repeats bits to all ports	One collision domain, one broadcast domain
Switch	2	MAC address table (learns source MACs)	Each port its own collision domain; one broadcast domain (without VLANs)
Router	3	IP routing table	Separates broadcast domains
Gateway	Up to 7	Protocol translation between networks	

A switch learns by noting each frame's source MAC and port; unknown destinations are **flooded**.

Q4. MAC address vs IP address.

MAC: 48-bit hardware address burned into the NIC, flat, used within one local network (link layer), changes at every hop. IP: logical address assigned by network, hierarchical (network + host), used end to end for routing. Analogy: MAC is your name, IP is your postal address. **ARP** maps an IP to a MAC on the local network: broadcast "who has 192.168.1.5?" and the owner replies with its MAC (cached). ARP spoofing is a local man-in-the-middle attack.

Q5. Error detection: parity, checksum, CRC, and Hamming code.

Parity bit detects odd numbers of bit errors. Internet checksum (one's-complement sum) is used by IP/TCP/UDP. **CRC** (polynomial division) detects burst errors and is used in Ethernet. Hamming codes can **correct** a single-bit error (parity bits at positions 1, 2, 4, 8...). Minimum Hamming distance d detects d-1 errors and corrects floor((d-1)/2).

CRC dry run: data `1101011011`, generator `10011` (degree 4), append 4 zeros: `11010110110000`, divide (XOR) by `10011` to get remainder `1110`; transmit `11010110111110`. The receiver divides the whole frame; remainder 0 means no detected error.

Q6. Flow control: stop-and-wait vs sliding window (Go-Back-N, Selective Repeat). Efficiency numerical.

Stop-and-wait: send one frame, wait for ACK. Efficiency = T_transmit / (T_transmit + 2 x T_propagation). **Sliding window** keeps several frames in flight. **Go-Back-N**: sender window N, receiver window 1; on an error retransmit the lost frame and all after it. **Selective Repeat**: both windows N; only the lost frame is retransmitted (receiver buffers out-of-order). Sequence-number requirements: GBN needs window <= 2^k - 1; SR needs window <= 2^(k-1).

Numerical: 1000-bit frame on a 1 Mbps link: T_t = 1 ms. One-way propagation 4 ms, so RTT part = 8 ms. Stop-and-wait efficiency = 1 / (1 + 8) = **11.1%**. Window size to fully use the link: 1 + 2 x T_p / T_t = **9 frames**.

Q7. Medium access: CSMA/CD, CSMA/CA, ALOHA.

Pure ALOHA: transmit any time (max efficiency 18.4%); slotted ALOHA 36.8%. **CSMA/CD** (wired Ethernet): listen first, detect collisions while sending, back off exponentially; min frame length must be >= 2 x propagation delay x bandwidth. **CSMA/CA** (Wi-Fi): collisions cannot be detected over the air, so avoid them with random backoff and RTS/CTS and ACKs. Modern switched full-duplex Ethernet has no collisions.

Q8. VLAN, STP, and what is a collision domain?

A **VLAN** logically splits one physical switch into separate broadcast domains (802.1Q tags). **Spanning Tree Protocol** prevents switching loops by blocking redundant links. A collision domain is the set of devices whose transmissions can collide.

Edge cases and gotchas

- Switch flooding: the first frame to an unknown MAC goes out of all ports except the ingress.
- MAC addresses change at every hop; IP addresses (without NAT) stay the same end to end.
- Routers do not forward broadcast frames; switches do.
- Ethernet minimum frame is 64 bytes, maximum payload (MTU) is 1500 bytes.
- Wi-Fi hidden-terminal problem is why RTS/CTS exists.
- Go-Back-N receiver discards out-of-order frames; selective repeat buffers them.

Rapid-fire round

Question	Answer
Layer of switch / router	2 / 3
Ethernet MTU	1500 bytes
MAC length	48 bits
Protocol mapping IP to MAC	ARP
CRC used in	Ethernet frames (link layer)
Which layer does TCP live in	Transport (4)
SR window limit	2^(k-1)

Question	Answer
ALOHA max efficiency	18.4% pure, 36.8% slotted

IP Addressing, Subnetting and Routing

> Most asked: IPv4 classes and CIDR, subnetting numericals, NAT, DHCP, ICMP/traceroute, IPv4 vs IPv6, routing algorithms (distance vector vs link state), BGP/OSPF/RIP.

Q1. IPv4 basics: address classes, private ranges, special addresses.

32-bit address in dotted decimal. Classful (historic): A 1-126 (/8), B 128-191 (/16), C 192-223 (/24), D multicast, E reserved. Modern: **CIDR** (`a.b.c.d/n`, n = network bits). Private ranges (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. Loopback `127.0.0.0/8`. Link-local `169.254.0.0/16` (failed DHCP). `0.0.0.0` unspecified; `255.255.255.255` limited broadcast.

Q2. Subnetting: worked numericals.

Given 192.168.10.77/26. Mask /26 = 255.255.255.192. Block size = 256 - 192 = 64. Subnets of the last octet: 0, 64, 128, 192. 77 lies in the block starting at 64.

- Network address: **192.168.10.64**; broadcast: **192.168.10.127**; usable hosts: .65 to .126.
- Hosts per subnet = 2^(32-26) - 2 = 2^6 - 2 = **62**.

Split 192.168.1.0/24 into 4 equal subnets: borrow 2 bits, new prefix /26, each 64 addresses (62 hosts): 192.168.1.0/26, .64/26, .128/26, .192/26.

VLSM: need 100, 50, 20 hosts from 192.168.1.0/24. Assign largest first: 100 hosts need 7 host bits (/25): 192.168.1.0/25 (0-127). 50 hosts need 6 bits (/26): 192.168.1.128/26 (128-191). 20 hosts need 5 bits (/27): 192.168.1.192/27 (192-223).

Route summarisation (supernetting): 192.168.0.0/24 + 192.168.1.0/24 + 192.168.2.0/24 + 192.168.3.0/24 aggregate to **192.168.0.0/22**.

Quick check "same subnet?": AND each IP with the mask; equal results mean same network.

Q3. What does the IPv4 header contain? Fragmentation?

Fields: version, header length, ToS/DSCP, total length, identification/flags/fragment offset (fragmentation), **TTL**, protocol (6 = TCP, 17 = UDP, 1 = ICMP), header checksum, source and destination IP. When a packet exceeds a link's **MTU**, routers fragment it (offset in 8-byte units); reassembly happens only at the destination. Example: 4000-byte datagram (20 header + 3980 data) over MTU 1500: fragments carry 1480 data bytes each: 1480 + 1480 + 1020; offsets 0, 185, 370; the last has MF = 0. IPv6 routers do not fragment (sender uses Path MTU Discovery).

Q4. IPv4 vs IPv6.

IPv6 has 128-bit addresses (hex, e.g. `2001:db8::1`), no broadcast (multicast/anycast), simpler fixed header, no router fragmentation, built-in autoconfiguration (SLAAC) and IPsec support, no checksum in the header. Transition: dual stack, tunnelling, NAT64.

Q5. NAT, DHCP, ICMP.

NAT/PAT: a router rewrites private source IP and port to its public IP and a unique port, keeping a translation table; saves IPv4 addresses but breaks end-to-end connectivity (hard for inbound connections, peer-to-peer). **DHCP** automatically assigns IP, mask, gateway, DNS via **DORA**: Discover (broadcast), Offer, Request, Acknowledge (over UDP 67/68). **ICMP** carries errors/diagnostics: `ping` (echo request/reply), `traceroute` (sends probes with increasing TTL; each router that drops TTL=0 replies "time exceeded").

Q6. Routing algorithms: distance vector vs link state.

	Distance Vector (RIP)	Link State (OSPF)
Knowledge	Neighbours' distance tables	Full topology map
Algorithm	Bellman-Ford	Dijkstra
Updates	Periodic, to neighbours	Flooded on change
Problem	Count-to-infinity, slow convergence	More CPU/memory

Count-to-infinity fixes: split horizon, poison reverse, max hop count (RIP: 15). **Intra-domain**: RIP, OSPF, IS-IS. **Inter-domain**: **BGP** (path-vector, policy-based, runs the Internet between autonomous systems, over TCP port 179).

Q7. Static vs dynamic routing; how does a router decide? Longest prefix match.

The router picks the routing-table entry with the **longest matching prefix**. Example table: `10.0.0.0/8 via A`, `10.1.0.0/16 via B`, default `0.0.0.0/0 via C`. Destination 10.1.2.3 matches /8 and /16, so goes via **B**; 10.9.9.9 goes via A; 8.8.8.8 via the default route C.

Edge cases and gotchas

- Usable hosts exclude the network and broadcast addresses (/31 and /32 are special cases).
- A host talks to another subnet via its **default gateway**: it ARPs the gateway's MAC, not the remote host's.
- TTL prevents routing loops: each hop decrements it.
- NAT hides internal hosts but is not a firewall.
- ICMP is carried inside IP but is not a transport protocol.
- `0.0.0.0/0` is the default route.
- IPv6 `::1` is loopback, `fe80::/10` link-local.

Rapid-fire round

Question	Answer
Hosts in a /24	254
Private IP ranges	10/8, 172.16/12, 192.168/16
DHCP steps	Discover, Offer, Request, Ack
Traceroute relies on	TTL expiry + ICMP time exceeded
OSPF algorithm	Dijkstra (link state)
RIP max hops	15
BGP transport	TCP port 179
IPv6 length	128 bits

Transport Layer: TCP and UDP

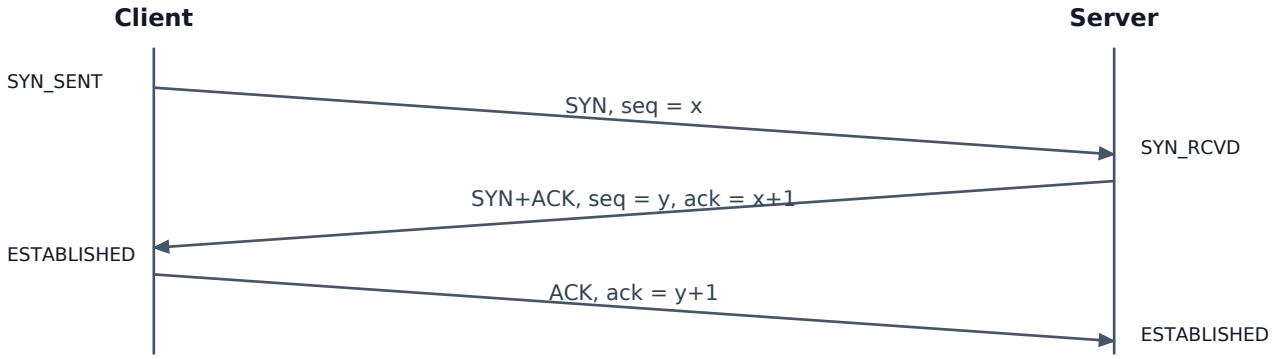
> Most asked: TCP vs UDP, three-way handshake, four-way close, TIME_WAIT, sequence/ack dry run, flow control vs congestion control, slow start, retransmission, ports and sockets.

Q1. TCP vs UDP.

	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	ACKs, retransmission, ordering	None
Flow/congestion control	Yes	No
Header	20+ bytes	8 bytes
Speed	Slower	Faster, low overhead
Use	Web, email, file transfer, SSH	DNS, video/voice, gaming, DHCP, QUIC base

UDP is message-oriented (boundaries preserved); TCP is a byte stream (no message boundaries, so applications must frame messages).

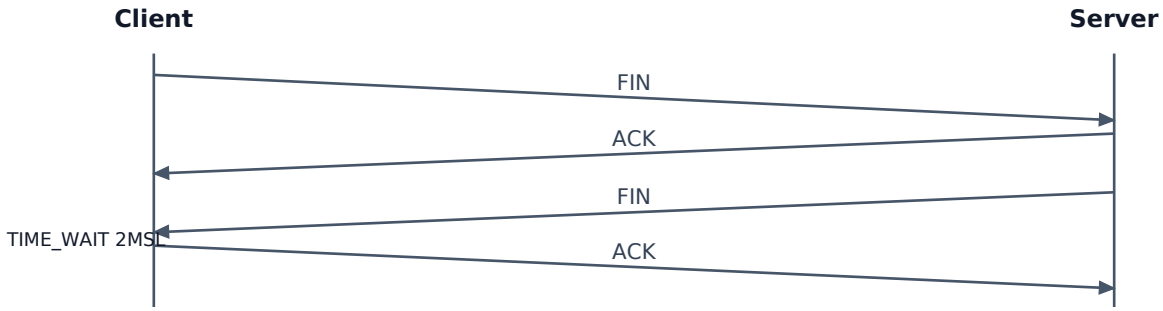
Q2. Three-way handshake and why three?



Purpose: both sides agree on **initial sequence numbers** and confirm that each can send and receive. Two steps cannot confirm the client's ability to receive nor protect against old duplicate SYNs. **SYN flood** attack: many SYNs fill the half-open queue; mitigated with SYN cookies.

Q3. Connection termination: four-way close and TIME_WAIT.

Each direction is closed independently: `FIN` (client) / `ACK` / `FIN` (server) / `ACK`. After sending the final ACK the active closer enters **TIME_WAIT** for 2 x MSL (about 60 s) so that the last ACK can be retransmitted if lost and old duplicate segments expire. Many TIME_WAIT sockets on a busy client/server are normal; port exhaustion is the risk. `RST` aborts a connection abruptly.



Q4. How does TCP provide reliability? Dry run of sequence and ack numbers.

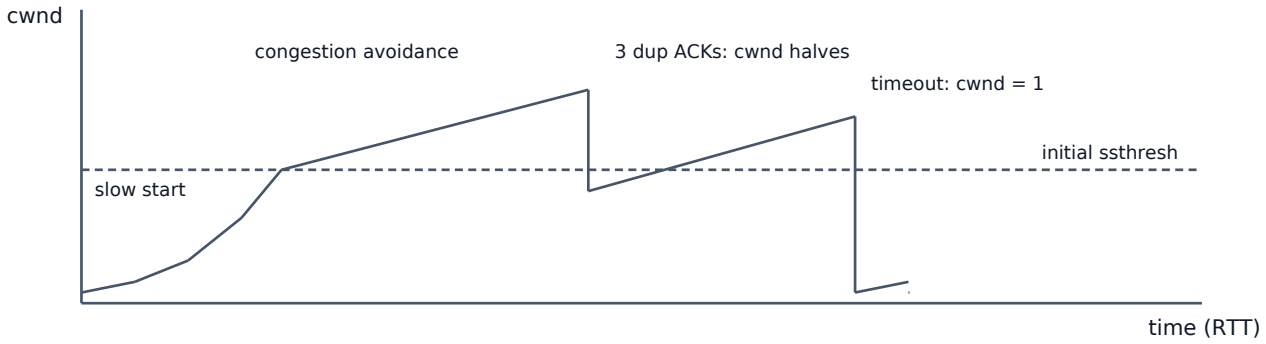
Sequence numbers (bytes), cumulative ACKs, timeouts and retransmission (RTO from measured RTT), checksums, **fast retransmit** (3 duplicate ACKs trigger retransmit without waiting), reordering at the receiver, SACK.

Dry run: client sends segments of 100 bytes starting at seq 1000: seg1 (1000-1099), seg2 (1100-1199), seg3 (1200-1299). Server receives seg1: ACK **1100** (next expected byte). seg2 is lost, seg3 arrives: server sends ACK **1100** again (duplicate). More out-of-order data brings more duplicate ACK 1100; after three duplicates the client retransmits seg2, server then ACKs **1300**.

Q5. Flow control vs congestion control.

Flow control protects the *receiver*: the receiver advertises a **window size (rwnd)** in each ACK; the sender never has more unacknowledged data than rwnd. **Congestion control** protects the *network*: the sender keeps a **congestion window (cwnd)**; it sends min(cwnd, rwnd).

- **Slow start**: cwnd starts at 1 MSS (modern: 10), doubles each RTT (+1 per ACK) until it reaches `ssthresh`.
- **Congestion avoidance** (AIMD): +1 MSS per RTT (additive increase).
- On loss by **timeout**: ssthresh = cwnd/2, cwnd = 1, back to slow start. On loss by **3 duplicate ACKs** (fast recovery/Reno): ssthresh = cwnd/2, cwnd = ssthresh, continue in avoidance (multiplicative decrease).



Variants: Reno, CUBIC (Linux default), BBR (model-based). Zero-window: receiver advertises 0, sender probes periodically.

Q6. Ports, sockets, and how a server handles many clients.

A **port** (16-bit) identifies a process on a host. A connection is identified by the **4-tuple** (src IP, src port, dst IP, dst port), so one server port serves many clients. Well-known: 20/21 FTP, 22 SSH, 25 SMTP, 53 DNS, 67/68 DHCP, 80 HTTP, 110 POP3, 143 IMAP, 443 HTTPS, 3306 MySQL, 5432 Postgres. Server socket calls: `socket, bind, listen, accept, recv/send, close`; client: `socket, connect, send/recv, close`. `accept` returns a **new socket** per connection; the listening socket stays open. Servers scale with threads, `select/poll/epoll` event loops, or async I/O.

Q7. Nagle's algorithm, delayed ACK, keepalive, head-of-line blocking, QUIC.

Nagle batches small writes (disable with `TCP_NODELAY` for latency-sensitive apps). Delayed ACK waits briefly to piggyback ACKs; Nagle plus delayed ACK can add ~40 ms stalls. Keepalive probes detect dead peers. **Head-of-line blocking**: one lost TCP segment stalls all later data on that connection (a problem for HTTP/2 multiplexing). **QUIC** (HTTP/3) runs over UDP, with independent streams, built-in TLS 1.3 and faster handshakes.

Edge cases and gotchas

- TCP ACK numbers name the *next byte expected*, not the last received.
- A SYN consumes one sequence number; so does a FIN.
- Closing a socket does not guarantee unsent data was delivered; use shutdown/graceful close.
- UDP has no congestion control, so heavy UDP can starve TCP; apps must self-limit.
- Server restart `Address already in use` is due to TIME_WAIT; `SO_REUSEADDR` helps.

- A TCP connection can be half-open if a peer crashes silently.
- Window size field is 16 bits (max 64 KB) unless the window-scaling option is used.

Rapid-fire round

Question	Answer
Handshake flags	SYN, SYN+ACK, ACK
TIME_WAIT lasts	2 x MSL
Flow control by	Receiver window (rwnd)
Congestion control by	cwnd (slow start, AIMD)
DNS transport	UDP 53 (TCP for large/zone transfers)
Fast retransmit trigger	3 duplicate ACKs
TCP header min size	20 bytes
UDP header size	8 bytes
Socket identified by	4-tuple (src/dst IP and port)

Application Layer: DNS, HTTP, TLS and Security

> Most asked: what happens when you type a URL, DNS resolution, HTTP methods/status codes/versions, cookies vs sessions, HTTPS and the TLS handshake, REST vs RPC, firewalls and common attacks.

Q1. What happens when you type a URL in the browser and press Enter?

1. **Parse** the URL; check browser/HSTS cache and the HTTP cache.

2. **DNS lookup**: browser cache, OS cache, hosts file, then the recursive resolver (see next question) returns an IP.

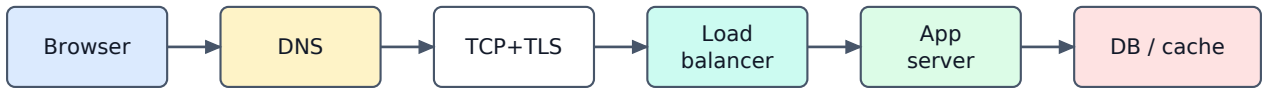
3. **ARP/routing**: the host decides the IP is off-subnet and sends frames to its **default gateway** (ARP for its MAC).

4. **TCP handshake** (SYN, SYN+ACK, ACK) to port 443 (or 80); packets are routed hop by hop (MAC rewritten each hop, IP preserved; NAT at the home router).

5. **TLS handshake** (for HTTPS): certificate verification and key exchange.

6. **HTTP request** (`GET / HTTP/1.1`, `Host`, cookies); load balancer/CDN/reverse proxy forwards to an app server, which may query a cache/database.

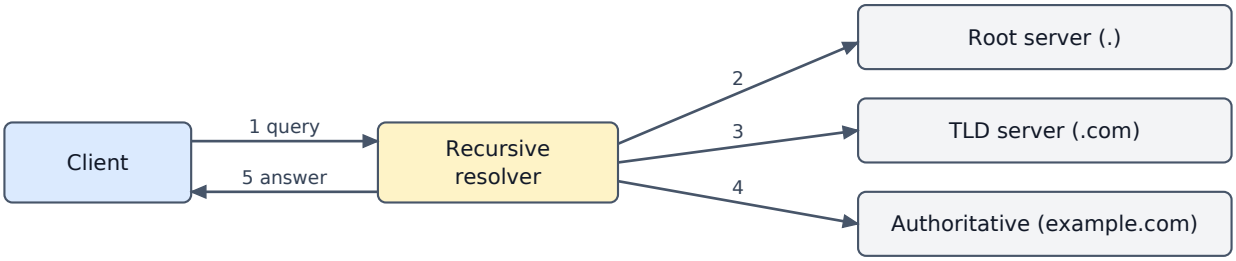
7. **Response** (status, headers, HTML); browser parses HTML, builds the DOM/CSSOM, fetches CSS/JS/images (more requests, possibly HTTP/2 multiplexed), runs JavaScript, renders, and may keep the connection alive.



request path; the response returns along the same path

Q2. How does DNS resolution work? Record types?

DNS maps names to IPs. Hierarchy: root (.), TLD (.com), authoritative server for the domain. A **recursive resolver** (ISP, 8.8.8.8) does the work: asks a root server (returns TLD servers), then the TLD server (returns the domain's authoritative servers), then the authoritative server (returns the answer), caching each result by **TTL**.



Records: **A** (IPv4), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail), **NS** (name server), **TXT** (SPF/verification), **PTR** (reverse), **SOA**. DNS uses UDP 53 (TCP for large responses/zone transfers), plus DoH/DoT for encryption. **Anycast** spreads root/CDN servers globally. Cache poisoning is mitigated by DNSSEC and randomised ports/IDs.

Q3. HTTP basics: methods, status codes, headers.

Request: method + path + headers + optional body. Methods: `GET` (read, safe, idempotent), `POST` (create/action, not idempotent), `PUT` (replace, idempotent), `PATCH` (partial), `DELETE` (idempotent), `HEAD`, `OPTIONS` (CORS preflight). Status codes: 1xx info; 2xx success (200, 201, 204); 3xx redirect (301 permanent, 302 temporary, 304 not modified); 4xx client error (400, 401 unauthenticated, 403 forbidden, 404, 405, 409, 429); 5xx server error (500, 502, 503, 504). Key headers: `Host`, `Content-Type`, `Content-Length`, `Authorization`, `Cookie/Set-Cookie`, `Cache-Control`, `ETag`, `User-Agent`, `Location`.

Q4. HTTP/1.1 vs HTTP/2 vs HTTP/3.

	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC (UDP)
Format	Text	Binary frames	Binary
Concurrency	Persistent connection, but one request at a time (HOL blocking), browsers open ~6 connections	**Multiplexing** many streams on one connection, header compression (HPACK), server push	Independent streams, no TCP HOL blocking
TLS	Optional	Practically required	Built in (TLS 1.3)

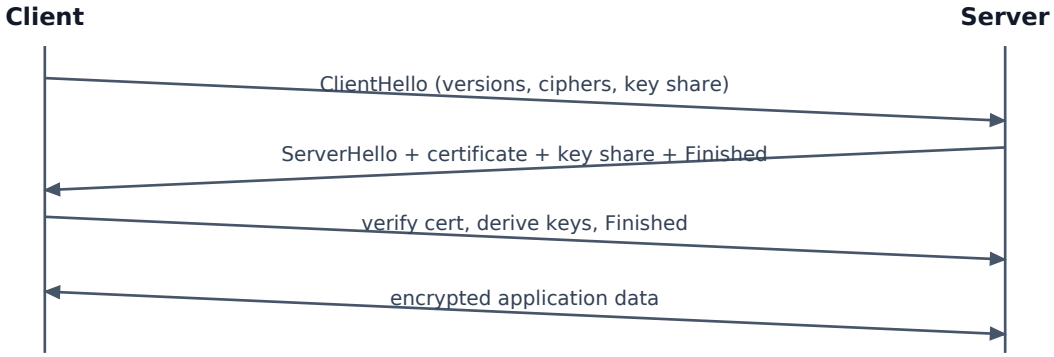
HTTP is **stateless**: each request is independent; state comes from cookies/tokens.

Q5. Cookies vs sessions vs tokens (JWT). HTTP caching.

Cookie: small data the server asks the browser to store and send back (`Set-Cookie`); flags `HttpOnly` (no JS access), `Secure` (HTTPS only), `SameSite` (CSRF defence), `Expires/Max-Age`. **Session**: server stores state keyed by a session ID held in a cookie. **JWT**: signed self-contained token. Caching: `Cache-Control: max-age=3600`, validators `ETag`/`Last-Modified` with conditional requests (`If-None-Match`) answered by `304 Not Modified`; CDNs cache static content close to users.

Q6. HTTPS and the TLS handshake. Symmetric vs asymmetric encryption.

Asymmetric (RSA, ECDSA, ECDH) is slow but solves key exchange and authentication; symmetric (AES, ChaCha20) is fast for bulk data. TLS uses asymmetric crypto to authenticate and agree on a **session key**, then symmetric for traffic.



Steps (TLS 1.3, 1 RTT): hello messages negotiate version and cipher, **ephemeral Diffie-Hellman** shares give forward secrecy, the server sends its **certificate** (public key signed by a Certificate Authority); the client validates the chain up to a trusted root, checks the domain name, validity and revocation; both derive session keys. TLS 1.2 needs 2 RTT; **session resumption/0-RTT** reduces latency. A certificate proves "this public key belongs to example.com"; **digital signature** = hash encrypted with the private key, verified with the public key.

Q7. Common web and network attacks and defences.

Attack	What	Defence
SQL injection	Untrusted input changes a query	Parameterised queries
XSS	Injected script runs in victim's browser	Output escaping, CSP, HttpOnly cookies
CSRF	Forged request using victim's cookies	SameSite cookies, CSRF tokens

Attack	What	Defence
MITM	Attacker intercepts traffic	TLS with certificate validation
DoS/DDoS	Exhaust resources	Rate limiting, CDN/WAF, SYN cookies
ARP/DNS spoofing	Poison caches	Static ARP/DAI, DNSSEC
Phishing	Social engineering	Awareness, MFA
Password attacks	Brute force, credential stuffing	Hashing (bcrypt/argon2) + salt, MFA, lockouts

Firewall: filters traffic by rules (stateless packet filter by IP/port; stateful tracks connections; application/WAF inspects content). **VPN** tunnels traffic encrypted over a public network. Hashing is one-way, encryption is reversible with a key.

Q8. Other application protocols: SMTP/IMAP/POP3, FTP, SSH, WebSocket, REST, gRPC.

Email: **SMTP** sends (25/587), **POP3** downloads and typically deletes (110), **IMAP** syncs mailboxes (143). **FTP** uses separate control (21) and data (20) connections, unencrypted; SFTP/FTPS secure it. **SSH** (22) is an encrypted remote shell with key-based auth. **WebSocket** upgrades an HTTP connection into a persistent full-duplex channel. **REST**: stateless resource-oriented APIs over HTTP; **gRPC**: HTTP/2 + Protobuf, binary and fast. **Long polling/SSE/WebSocket** are server-push options.

Q9. Load balancer, reverse proxy, CDN, forward proxy.

A **forward proxy** acts for clients (filtering, anonymity). A **reverse proxy** sits in front of servers (TLS termination, caching, compression, routing; Nginx). A **load balancer** spreads requests across servers (round robin, least connections, IP hash) and does health checks; L4 balances on IP/port, L7 on HTTP content. A **CDN** caches content at edge locations near users.

Q10. Bandwidth vs latency vs throughput; RTT; delays.

Bandwidth: link capacity. **Latency**: time for a bit to travel. **Throughput**: actual rate achieved. Delay components: **transmission** (L/R = packet size / rate), **propagation** (distance / speed), **processing**, **queuing**. Numerical: 1500-byte packet on 10 Mbps: 12000 bits / 10^7 = 1.2 ms transmission; over 3000 km at 2x10^8 m/s: 15 ms propagation. **Bandwidth-delay product** = bandwidth x RTT = amount of data in flight to fill the pipe (relevant to TCP window sizing). Circuit vs packet switching: reserved path vs statistical sharing (the Internet uses packet switching).

Edge cases and gotchas

- `301` vs `302`: permanent (cached, SEO transfers) vs temporary.
- `401` means unauthenticated, `403` means authenticated but not allowed.
- HTTPS hides the path and body but not the destination IP (and SNI may reveal the hostname unless ECH is used).
- A self-signed certificate encrypts but does not authenticate the server.
- GET requests must be safe (no side effects); crawlers and prefetchers call them.
- DNS TTL means changes propagate gradually.
- CORS is enforced by browsers, not servers.
- Connection reuse (keep-alive) avoids repeated handshakes.
- Hashing passwords with plain SHA-256 is insufficient; use slow, salted hashes.

Rapid-fire round

Question	Answer
DNS port	53
HTTPS default port	443
Stateless means	Server keeps no client context between requests
Idempotent methods	GET, PUT, DELETE, HEAD, OPTIONS

Question	Answer
502 vs 504	Bad response from upstream vs upstream timeout
Symmetric key used for	Bulk data after TLS handshake
CA role	Vouches that a public key belongs to a domain
CNAME	Alias to another name
SameSite cookie protects against	CSRF
Load balancer layers	L4 (TCP) and L7 (HTTP)

Part D — DBMS and SQL

Order: data models, ER and keys, normalisation (with dry runs), transactions and concurrency, indexing and storage, then SQL with one sample schema where **every query shows its verified output**, and finally NoSQL and scaling basics.

DBMS Basics, ER Model and Keys

> Most asked: DBMS vs file system, three-schema architecture, keys (super, candidate, primary, foreign), ER to tables, constraints, DDL/DML/DCL/TCL.

Q1. What is a DBMS? Advantages over a file system?

Software to define, store, query and manage data. Advantages: reduced redundancy and inconsistency, **data independence**, concurrent access with isolation, integrity constraints, security, backup/recovery, standard query language. Cost: overhead and complexity for tiny, single-user data.

Q2. Three-schema architecture and data independence.

Physical (how data is stored: files, indexes), **logical/conceptual** (tables, relationships, constraints for the whole database), **view/external** (what each user sees). **Physical data independence**: change storage/indexes without changing the logical schema. **Logical data independence**: change the logical schema without breaking views/applications (harder).

Q3. Keys: define each with an example.

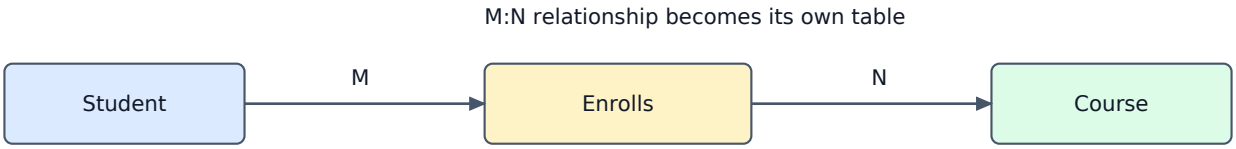
Table `Employee(emp_id, email, name, phone, dept_id)`.

- **Super key**: any set of attributes that uniquely identifies a row (`{emp_id}`, `{emp_id, name}`, `{email, phone}`).
- **Candidate key**: a *minimal* super key (`{emp_id}`, `{email}`).
- **Primary key**: the chosen candidate key; unique and NOT NULL, one per table.
- **Alternate key**: candidate keys not chosen as primary (`email`).
- **Composite key**: more than one attribute (`(student_id, course_id)`).
- **Foreign key**: attribute referencing another table's primary/unique key; enforces **referential integrity** (`dept_id` references `Department`).
- **Surrogate key**: system-generated (auto-increment, UUID) with no business meaning; **natural key** has meaning (email).

Unique allows one or more NULLs (DB-dependent); primary key allows none.

Q4. ER model: entities, relationships, cardinality, and mapping to tables.

Entity (Student), **attribute** (simple, composite, multivalued, derived), **relationship** (Enrolls), **weak entity** (needs an owner's key, e.g. Dependent of Employee). Cardinality: 1:1, 1:N, M:N. Participation: total or partial.



Mapping rules: each strong entity becomes a table; **1:N**: put the "one" side's key as a foreign key in the "many" table; **M:N**: create a junction table with both keys as a composite primary key plus relationship attributes; **multivalued** attribute: separate table; **weak entity**: table with owner's key + partial key as the primary key.

```
CREATE TABLE enrolls (student_id INT REFERENCES student(student_id),
                      course_id  INT REFERENCES course(course_id),
                      grade CHAR(2), PRIMARY KEY (student_id, course_id));
```

Q5. Integrity constraints; referential actions.

`NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK (salary > 0)`, `DEFAULT`. Foreign key actions on parent delete/update: `CASCADE`, `SET NULL`, `SET DEFAULT`, `RESTRICT`/`NO ACTION` (reject). Entity integrity: no NULL in primary key. Referential integrity: a foreign key value must exist in the parent (or be NULL).

Q6. DDL vs DML vs DCL vs TCL; DELETE vs TRUNCATE vs DROP.

DDL defines structure (`CREATE, ALTER, DROP, TRUNCATE`); **DML** manipulates data (`SELECT, INSERT, UPDATE, DELETE`); **DCL** controls access (`GRANT, REVOKE`); **TCL** controls transactions (`COMMIT, ROLLBACK, SAVEPOINT`).

	DELETE	TRUNCATE	DROP
Removes	Selected rows (WHERE)	All rows	Table itself
Logging / speed	Row by row, slower	Minimal, fast	
Triggers	Fire	Don't	
Rollback	Yes	Depends (PostgreSQL yes, MySQL no)	Depends
Resets identity	No	Usually yes	

Q7. Relational algebra basics.

Selection `sigma` (filter rows), projection `pi` (choose columns, removes duplicates), union, intersection, difference, Cartesian product, join (theta, equi, natural), rename, division. Example: `pi name (sigma salary>60000 (Employee))` equals `SELECT DISTINCT name FROM Employee WHERE salary > 60000`.

Edge cases and gotchas

- A table can have many candidate keys but only one primary key.
- A foreign key can reference a UNIQUE column, not only the primary key.
- Foreign keys can be NULL (the row simply has no parent).
- `CASCADE` delete can remove huge amounts of data silently.
- UUID primary keys avoid collisions across systems but fragment B-tree inserts compared to sequential IDs.
- Relation = set of tuples (no order, no duplicates in theory); SQL tables allow duplicates unless constrained.

Rapid-fire round

Question	Answer
Candidate key	Minimal super key
Primary key NULLs	Not allowed
Foreign key purpose	Referential integrity
M:N relationship implemented as	Junction table
Weak entity key	Owner's key + partial key
DML example	INSERT, UPDATE, DELETE, SELECT
Data independence	Change a lower level without changing the level above

Normalisation and Functional Dependencies

> Most asked: anomalies, functional dependencies, finding candidate keys by closure, 1NF to BCNF with decomposition dry runs, 3NF vs BCNF, denormalisation.

Q1. Why normalise? What are the anomalies?

Normalisation removes redundancy to avoid anomalies. In `Orders(order_id, customer_name, customer_city, product, qty)` repeating customer data causes: **update anomaly** (change a city in one row only, leaving inconsistent data), **insertion anomaly** (cannot add a customer without an order), **deletion anomaly** (deleting a customer's last order loses the customer).

Q2. Functional dependency, closure, and finding candidate keys (dry run).

`X -> Y` means whenever two rows agree on X they agree on Y. **Armstrong's axioms**: reflexivity (`Y subset X` gives `X -> Y`), augmentation, transitivity; derived: union, decomposition, pseudo-transitivity. The **closure X+** is all attributes determined by X.

Example: R(A, B, C, D, E) with FDs `A -> B`, `BC -> D`, `D -> E`.

- A+ = {A, B}. Not a key.
- AC+ = {A, C}, then B from A, so {A, B, C}, then D from BC, then E from D: {A, B, C, D, E}. So **AC is a key**.
- Is A or C alone a key? A+ = {A,B}, C+ = {C}: no. So **AC is the only candidate key** (A and C never appear on a right side, so they must be in every key).

Shortcut: attributes that never appear on a right-hand side must be in every key; start from them and add attributes until the closure covers R.

Q3. Define 1NF, 2NF, 3NF, BCNF with a worked decomposition.

- **1NF**: every attribute is atomic (no repeating groups or lists in a cell).
- **2NF**: 1NF + no **partial dependency** (a non-prime attribute depending on *part* of a composite key).
- **3NF**: 2NF + no **transitive dependency** (non-prime attribute depending on another non-prime attribute). Formal: for every FD `X -> Y`, X is a superkey **or** Y is a prime attribute.
- **BCNF**: for every non-trivial FD `X -> Y`, **X is a superkey**.

Worked example: `Enrollment(student_id, course_id, student_name, course_name, instructor, instr_phone)`, key `(student_id, course_id)`; FDs: `student_id -> student_name`, `course_id -> course_name, instructor`, `instructor -> instr_phone`.

1. 1NF: already atomic.

2. 2NF: `student_name` depends on part of the key and `course_name`, `instructor` depend on part of the key. Split: `Student(student_id, student_name)`, `Course(course_id, course_name, instructor, instr_phone)`, `Enrollment(student_id, course_id)`.

3. 3NF: in `Course`, `instructor -> instr_phone` is transitive (`course_id -> instructor -> instr_phone`). Split: `Course(course_id, course_name, instructor)` and `Instructor(instructor, instr_phone)`.

Final: `Student`, `Course`, `Instructor`, `Enrollment`: all in 3NF (and BCNF).

Q4. 3NF vs BCNF: show a table in 3NF but not BCNF.

`Teaches(student, course, teacher)` with FDs `(student, course) -> teacher` and `teacher -> course` (each teacher teaches one course). Keys: `(student, course)` and `(student, teacher)`. In `teacher -> course`, `teacher` is not a superkey, so **not BCNF**, but `course` is a prime attribute so it **is 3NF**. BCNF decomposition: `R1(teacher, course)` and `R2(student, teacher)`; this loses the dependency `(student, course) -> teacher` (cannot be enforced without a join). That is the trade-off: **BCNF may not preserve dependencies; 3NF always can** (lossless and dependency-preserving).

Q5. Lossless join decomposition and dependency preservation.

Decomposing R into R1 and R2 is **lossless** if `R1 intersect R2` is a key of R1 or R2 (joining back gives exactly R, no spurious tuples). **Dependency preserving** if every FD can be checked within a single decomposed table. 4NF handles multivalued dependencies (`X ->> Y`); 5NF handles join dependencies.

Q6. Denormalisation: when and why?

Deliberately adding redundancy (precomputed totals, duplicated columns, materialised views) to speed reads and avoid joins in read-heavy or analytic systems (warehouses, caches). Cost: extra writes, risk of inconsistency, more storage. Normalise first; denormalise only after measuring.

Edge cases and gotchas

- If all candidate keys are single attributes, the table is automatically in 2NF.
- Every table with two attributes is in BCNF.
- Prime attribute = part of some candidate key.
- A table can have several candidate keys; check dependencies against all of them.
- Normal forms are about dependencies, not about the number of tables.
- Closure computation must iterate until nothing new is added.

Rapid-fire round

Question	Answer
1NF	Atomic values
2NF removes	Partial dependencies
3NF removes	Transitive dependencies
BCNF rule	Every determinant is a superkey
Always achievable losslessly and dependency-preserving	3NF
Lossless test	Common attributes form a key of one part
Armstrong axioms	Reflexivity, augmentation, transitivity

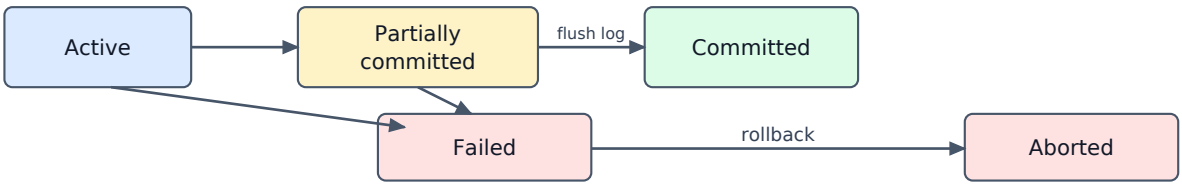
Transactions, Concurrency Control and Recovery

Most asked: ACID with examples, transaction states, isolation levels and anomalies, serializability and precedence graphs, 2PL, deadlock, MVCC, locks, WAL and recovery.

Q1. What is a transaction? Explain ACID with a bank transfer.

A transaction is a logical unit of work: a sequence of operations that must execute as a whole. Transfer 100 from A to B: `read(A); A = A - 100; write(A); read(B); B = B + 100; write(B)`.

- **Atomicity**: all or nothing (if the crash occurs after debiting A, undo it). Implemented with the log/rollback.
- **Consistency**: moves the database from one valid state to another (A + B stays constant, constraints hold).
- **Isolation**: concurrent transactions do not see each other's partial effects. Implemented by locking/MVCC.
- **Durability**: once committed, it survives crashes. Implemented by write-ahead log and flushing.



Q2. Concurrency problems and isolation levels.

Anomalies: **dirty read** (read uncommitted data of another transaction), **non-repeatable read** (same row read twice gives different values), **phantom read** (a repeated range query sees new rows), **lost update** (two writers overwrite each other), **write skew** (two transactions read overlapping data and write disjoint rows, breaking an invariant, e.g. two doctors both go off-call).

Isolation level	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	prevented	possible	possible
Repeatable Read	prevented	prevented	possible (standard); prevented in PostgreSQL/InnoDB via snapshots/gap locks
Serializable	prevented	prevented	prevented

Defaults: PostgreSQL and Oracle use Read Committed; MySQL InnoDB uses Repeatable Read. Higher isolation costs concurrency (more blocking or aborts).

Q3. Serializability: conflict graph dry run.

Two operations **conflict** if they are from different transactions, touch the same item, and at least one is a write (RW, WR, WW). A schedule is **conflict serializable** if its **precedence graph** (edge Ti to Tj when an operation of Ti conflicts with a *later* operation of Tj) has **no cycle**.

- S1: `R1(A) W1(A) R2(A) W2(A) R1(B) W1(B)`. Conflicts on A: W1(A) before R2(A) and W2(A), so edge T1 to T2. On B only T1 acts. Graph T1 to T2: **no cycle, serializable** (equivalent to T1 then T2).
- S2: `R1(A) R2(A) W1(A) W2(A)`. R1(A) before W2(A): T1 to T2. R2(A) before W1(A): T2 to T1. **Cycle, not conflict-serializable** (lost update: both read the old A).

Other notions: view serializability, **recoverable** schedules (a transaction commits only after those whose data it read commit), **cascadeless** (avoid cascading aborts), strict schedules.

Q4. Locking: shared/exclusive locks, 2PL, and deadlocks.

S (shared) lock to read (compatible with other S), **X (exclusive)** to write (compatible with none). **Two-Phase Locking (2PL)**: a transaction has a **growing** phase (acquire only) then a **shrinking** phase (release only); guarantees conflict serializability. **Strict 2PL** holds exclusive locks until commit, giving recoverable, cascadeless schedules (used in practice). 2PL can **deadlock** (T1 holds A wants B; T2 holds B wants A). Handling: detect with a wait-for graph cycle and abort a victim, use timeouts, or prevent via ordering/wait-die and wound-wait. Lock granularity: row, page, table (intention locks IS/IX allow hierarchical locking).

Q5. What is MVCC? Optimistic vs pessimistic concurrency.

Multi-Version Concurrency Control keeps multiple versions of rows with timestamps/transaction IDs; readers see a consistent **snapshot** and never block writers (and vice versa); old versions are cleaned later (PostgreSQL VACUUM, InnoDB undo purge). **Pessimistic** concurrency locks up front (`SELECT ... FOR UPDATE`); **optimistic** assumes few conflicts, checks at commit (version column: `UPDATE ... WHERE id = ? AND version = ?`, retry if 0 rows changed).

```
BEGIN;
SELECT balance FROM account WHERE id = 1 FOR UPDATE;   -- locks the row until COMMIT
UPDATE account SET balance = balance - 100 WHERE id = 1;
COMMIT;
```

Q6. Recovery: write-ahead logging, checkpoints, undo/redo.

WAL rule: log records must reach stable storage *before* the corresponding data page is written, and the commit record before acknowledging commit. After a crash (ARIES): **Analysis** (find dirty pages and active transactions from the last checkpoint), **Redo** (replay all logged changes to restore the state at crash), **Undo** (roll back uncommitted transactions). Checkpoints bound recovery time. Policies: *steal/no-force* (pages of uncommitted transactions may be flushed; commit does not force data pages) needs both undo and redo, and is what real systems use.

Q7. SAVEPOINT, autocommit, distributed transactions (2PC), CAP.

`SAVEPOINT s; ... ROLLBACK TO s;` rolls back part of a transaction. Many clients run in **autocommit** mode (each statement is its own transaction). **Two-Phase Commit** across databases: a coordinator asks all participants to *prepare* (vote), then tells everyone to commit or abort; blocking if the coordinator fails. **CAP theorem**: during a network partition a distributed system must choose consistency or availability. Sagas (compensating transactions) are an alternative in microservices.

Edge cases and gotchas

- A schedule can be serializable but not recoverable (commit order matters).
- Read Committed still allows non-repeatable reads: the same query twice in one transaction may differ.
- `SELECT ... FOR UPDATE` on rows that don't exist does not stop another insert (phantoms); use unique constraints or serializable isolation.
- Long transactions hold locks and bloat MVCC versions; keep them short.
- Deadlock victims need retry logic in the application.
- Lost update fix: atomic `UPDATE x = x - 100` instead of read-modify-write in the app.
- A unique constraint is the most reliable way to prevent duplicate inserts under concurrency.

Rapid-fire round

Question	Answer
ACID	Atomicity, Consistency, Isolation, Durability
Dirty read prevented from	Read Committed
Phantom prevented by	Serializable

Question	Answer
2PL phases	Growing then shrinking
Strict 2PL benefit	Cascadeless, recoverable
Deadlock detection structure	Wait-for graph
WAL means	Log before data
Default PostgreSQL isolation	Read Committed
Optimistic locking field	Version number / timestamp

Indexing, Storage and Query Processing

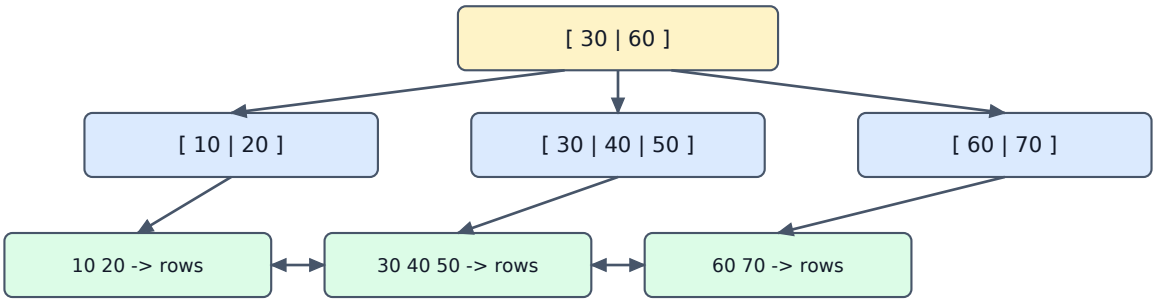
> Most asked: why indexes speed up queries, B-tree vs B+ tree, clustered vs non-clustered, composite index and the leftmost prefix, when indexes are not used, EXPLAIN, hashing, partitioning and sharding.

Q1. What is an index and why does it help?

An index is a separate sorted (or hashed) structure mapping column values to row locations, like a book's index. Without it, a query scans the whole table (**O(n)**); with a B+ tree it does an **O(log n)** lookup. Costs: extra storage and slower `INSERT/UPDATE/DELETE` (indexes must be maintained). Index columns used in `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`; do not index everything.

Q2. B-tree vs B+ tree. Why do databases use B+ trees?

In a **B+ tree**, internal nodes hold only keys (guides); **all records/pointers live in leaf nodes**, and leaves are linked in a sorted list. This gives high fanout (shallow tree), predictable lookups (always leaf depth), and efficient **range scans** (walk the leaf chain).



Height estimate: 4 KB page, 8-byte key plus 8-byte pointer gives fanout about 250. Height 3 holds about 250^3 = 15.6 million keys, so a lookup in a 10-million-row table costs about 3 to 4 page reads.

Q3. Clustered vs non-clustered index; covering index.

A **clustered index** defines the physical order of the table rows (the leaf level *is* the table): only one per table (InnoDB's primary key, SQL Server's default). A **non-clustered (secondary)** index stores key + pointer (row ID or primary key) in its own structure, requiring an extra lookup ("bookmark lookup") unless it is **covering** (the index contains all columns the query needs, an index-only scan). PostgreSQL tables are heap-organised; its indexes are all secondary (`CLUSTER` is a one-time reorder).

Q4. Composite index and the leftmost-prefix rule (examples).

Index on `(dept_id, salary)`. It serves: `WHERE dept_id = 2`, `WHERE dept_id = 2 AND salary > 50000`, `WHERE dept_id = 2 ORDER BY salary`. It does **not** efficiently serve `WHERE salary > 50000` alone (skips the first column). Put equality columns first, then range columns. Order matters.

Q5. When is an index NOT used?

- Function or expression on the column: `WHERE LOWER(name) = 'asha'` (fix: expression index).
- Leading wildcard: `LIKE '%sha'` (but `LIKE 'As%'` can use it).
- Type mismatch / implicit cast (`WHERE phone = 12345` for a text column).
- Low selectivity: boolean/status columns where most rows match, so a scan is cheaper.
- `OR` across different columns, `!=`, `NOT IN`, and leftmost-prefix violations.
- Tiny tables, or stale statistics making the planner choose a scan.

Always verify with `EXPLAIN (ANALYZE)`: look for **Index Scan/Seek** vs **Seq Scan/Table Scan**, estimated vs actual rows, and join type.

Q6. Hash index vs B+ tree; other index types.

Hash index: O(1) equality only, no range or ordering. B+ tree: equality, range, sorting, prefix. Others: bitmap (low-cardinality columns, warehouses), full-text/inverted (GIN), spatial (R-tree/GiST), partial (`WHERE active`), unique, covering (`INCLUDE`).

Q7. How does the database execute a query? Join algorithms.

Parse, rewrite, **optimise** (cost-based: use statistics to choose index/scan order and join method), execute. Join algorithms: **nested loop** (good for small outer + indexed inner), **hash join** (build a hash on the smaller table, probe with the larger; good for large unsorted equi-joins), **sort-merge join** (both inputs sorted; good for large sorted/indexed inputs). SQL logical order of evaluation: `FROM/JOIN` then `WHERE` then `GROUP BY` then `HAVING` then `SELECT` then `DISTINCT` then `ORDER BY` then `LIMIT`; that is why a `SELECT` alias cannot be used in `WHERE`.

Q8. Partitioning, sharding, replication.

Partitioning: split a big table into parts inside one database (range, list, hash) for pruning and maintenance. **Sharding**: distribute rows across multiple servers by a shard key (needs good key: high cardinality, even load; cross-shard joins and transactions are hard). **Replication**: copies for availability and read scaling (primary-replica, usually asynchronous so replicas can lag). Caching (Redis) and connection pooling reduce load too.

Edge cases and gotchas

- More indexes means slower writes and more disk; drop unused ones.
- Primary key lookups on random UUIDs fragment a clustered B+ tree.
- `COUNT(*)` on big tables is still a scan in MVCC databases.
- `SELECT *` defeats covering indexes.
- Stale statistics cause bad plans: `ANALYZE` / update statistics.
- Index on a column with NULLs: behaviour varies by DB (PostgreSQL indexes NULLs; Oracle does not index all-NULL entries).
- `OFFSET` pagination reads and discards rows; prefer keyset pagination.

Rapid-fire round

Question	Answer
Index structure used most	B+ tree
Clustered indexes per table	One
Leftmost prefix example	(a, b) serves a, and a+b, not b alone
Range query friendly	B+ tree, not hash
Covering index	Contains every column the query needs
LIKE 'abc%' vs '%abc'	Index usable vs not
Shard key quality	High cardinality, even distribution

SQL Fundamentals and Sample Schema

> Every example in this topic and the next uses the **same four tables**. Their contents are listed here so you can trace each query by hand. All outputs shown were produced by actually running the queries.

Q1. The sample schema and data (memorise this once).

```
departments(dept_id PK, dept_name)
  1 Engineering | 2 Sales | 3 HR | 4 Finance          (Finance has no employees)

employees(emp_id PK, name, dept_id FK, manager_id FK->emp_id, salary, hire_date)
  1 Asha   1     NULL  90000  2020-01-15
  2 Ben    1     1     70000  2021-03-01
  3 Chen   1     1     70000  2022-07-10
  4 Divya  2     NULL  80000  2019-05-20
  5 Esha   2     4     50000  2023-02-11
  6 Farhan 2     4     60000  2021-09-30
  7 Gita   3     NULL  55000  2020-11-05
  8 Hari   NULL  1     45000  2024-01-08               (no department)

customers(customer_id PK, name, city)
  1 Ann Delhi | 2 Raj Mumbai | 3 Sara Delhi | 4 Tom Pune   (Tom has no orders)

orders(order_id PK, customer_id FK, amount, order_date)
  101 1 500 2026-01-05 | 102 1 300 2026-02-10 | 103 2 700 2026-02-11
  104 3 200 2026-03-01 | 105 1 450 2026-03-15 | 106 2 100 2026-03-20
```

Edge data is deliberate: a department with no employees, an employee with no department, a tie in salary (Ben and Chen), a customer with no orders, and NULLs.

Q2. Write the DDL for these tables. Which data types and constraints matter?

```
CREATE TABLE departments (
  dept_id   INT PRIMARY KEY,
  dept_name VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE employees (
  emp_id     INT PRIMARY KEY,
  name       VARCHAR(50) NOT NULL,
  dept_id    INT REFERENCES departments(dept_id) ON DELETE SET NULL,
  manager_id INT REFERENCES employees(emp_id),
  salary     NUMERIC(10,2) CHECK (salary > 0),
  hire_date  DATE DEFAULT CURRENT_DATE
);
CREATE INDEX idx_emp_dept_salary ON employees(dept_id, salary);
ALTER TABLE employees ADD COLUMN email VARCHAR(100) UNIQUE;
```

Use `NUMERIC/DECIMAL` for money (never `FLOAT`); `CHAR` is fixed-length, `VARCHAR` variable; `TIMESTAMP` vs `DATE`; `BOOLEAN`; `JSONB` in PostgreSQL.

Q3. INSERT, UPDATE, DELETE, and upsert. Safe patterns.

```
INSERT INTO departments (dept_id, dept_name) VALUES (5, 'Legal');
UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 1;      -- always have a WHERE
DELETE FROM employees WHERE hire_date < '2020-01-01';
-- upsert (PostgreSQL): insert or update on conflict
INSERT INTO departments (dept_id, dept_name) VALUES (5, 'Legal Affairs')
ON CONFLICT (dept_id) DO UPDATE SET dept_name = EXCLUDED.dept_name;
-- MySQL:  INSERT ... ON DUPLICATE KEY UPDATE ...
```

Run a `SELECT` with the same `WHERE` first, and wrap risky changes in a transaction you can `ROLLBACK`.

Q4. WHERE vs HAVING; the logical order of a query.

`WHERE` filters **rows before** grouping and cannot use aggregates; `HAVING` filters **groups after** aggregation. Order: `FROM, JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT`. Every non-aggregated column in `SELECT` must appear in `GROUP BY` (strict databases enforce this).

```
SELECT dept_id, COUNT(*) AS c, AVG(salary) AS a
FROM employees GROUP BY dept_id HAVING COUNT(*) > 1 ORDER BY dept_id;
```

Output (Hari's NULL dept group has 1 row and Gita's dept has 1, so both are filtered out):

dept_id	c	a
1	3	76666.67
2	3	63333.33

Q5. NULL handling: the classic traps (with output).

`NULL` means unknown; any comparison with it yields UNKNOWN, treated as false in `WHERE`.

```
SELECT name FROM employees WHERE dept_id = NULL;      -- returns 0 rows (never true)
SELECT name FROM employees WHERE dept_id IS NULL;     -- Hari
SELECT COUNT(*), COUNT(dept_id), COUNT(DISTINCT dept_id) FROM employees;
```

Output of the last query: `8, 7, 3` (`COUNT(*)` counts rows, `COUNT(col)` skips NULLs, DISTINCT counts distinct non-NULL values). Tools: `COALESCE(dept_id, -1)` returns the first non-NULL; `NULLIF(a, b)` returns NULL if equal (avoids divide by zero: `x / NULLIF(y, 0)`). `NULL` in `GROUP BY` forms one group; `ORDER BY` puts NULLs first or last depending on the database (`NULLS LAST`). `NOT IN` with a NULL in the list returns **no rows** at all (see the anti-join question).

Q6. Aggregate and scalar functions you should know.

Aggregates: `COUNT, SUM, AVG, MIN, MAX` (ignore NULLs except `COUNT(*)`). Strings: `UPPER, LOWER, TRIM, SUBSTRING, CONCAT, LENGTH, REPLACE`, `LIKE` (`%` any string, _ one char). Dates: `CURRENT_DATE, EXTRACT(MONTH FROM d), DATE_TRUNC('month', d)`, date arithmetic. `CASE WHEN ... THEN ... ELSE ... END` is SQL's if/else, usable in `SELECT`, `ORDER BY`, aggregates (conditional counting). `BETWEEN a AND b` is inclusive. `DISTINCT` removes duplicate rows. `LIMIT/OFFSET` (MySQL/PostgreSQL), `TOP` (SQL Server), `FETCH FIRST` (standard).

Q7. Views, stored procedures, triggers, functions, CTEs.

A **view** is a stored query (virtual table; hides complexity, restricts columns); a **materialised view** stores the result and needs refresh. A **stored procedure** is server-side code you call (can modify data, manage transactions); a **function** returns a value and is used inside queries. A **trigger** runs automatically on INSERT/UPDATE/DELETE (audit logs; can hide logic and hurt performance). A **CTE** (`WITH t AS (...)`) names a subquery for readability and recursion.

Edge cases and gotchas

- `WHERE salary = NULL` returns nothing; use `IS NULL`.
- `BETWEEN` includes both ends; with timestamps `BETWEEN '2026-03-01' AND '2026-03-31'` misses March 31 afternoon; use `>= start AND < next_start`.
- `UNION` removes duplicates (sorts/hashes); `UNION ALL` keeps them and is faster.
- `COUNT(column)` vs `COUNT(*)` differ when NULLs exist.
- Integer division: `5/2` is 2 in many databases; cast to decimal.
- `ORDER BY` is not guaranteed unless written; `LIMIT` without it is arbitrary.
- `DELETE` without `WHERE` deletes everything.

Rapid-fire round

Question	Answer
WHERE vs HAVING	Rows before grouping vs groups after
Which aggregate counts NULLs	Only COUNT(*)
`= NULL` result	Never true; use IS NULL
UNION vs UNION ALL	Dedupe vs keep duplicates
View	Stored query, not stored data (unless materialised)
Trigger	Auto-executed on data change
Money type	NUMERIC/DECIMAL

SQL Joins, Subqueries and Window Functions

All queries run against the sample schema from the previous topic. Outputs are exact.

Q1. Explain the joins with outputs.

Join	Rows kept from A (left)	Rows kept from B (right)
INNER	Only those with a match	Only those with a match
LEFT	All (NULLs for B when no match)	Only matches
RIGHT	Only matches	All (NULLs for A when no match)
FULL OUTER	All	All
CROSS	Every row paired with every row (n x m), no condition	

INNER JOIN: only matching rows. **LEFT JOIN**: all rows of the left table, NULLs where no match. **RIGHT**: mirror. **FULL OUTER**: all rows from both. **CROSS**: every combination (n x m). **SELF**: a table joined to itself.

```
SELECT d.dept_name, e.name FROM employees e INNER JOIN departments d ON e.dept_id = d.dept_id;
```

dept_name	name
Engineering	Asha, Ben, Chen (3 rows)
Sales	Divya, Esha, Farhan (3 rows)
HR	Gita

Hari (no department) and Finance (no employees) are **both missing**: 7 rows.

```
SELECT e.name, d.dept_name FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

Now all 8 employees appear; Hari's `dept_name` is `NULL`.

Q2. Find departments with no employees, and count employees per department including empty ones.

```
SELECT d.dept_name, COUNT(e.emp_id) AS n
FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name ORDER BY d.dept_name;
```

dept_name	n
Engineering	3
Finance	0
HR	1
Sales	3

Count `e.emp_id` (not `*`), otherwise Finance shows 1 (the NULL-extended row). To list only empty departments: add `WHERE e.emp_id IS NULL` (result: Finance). **LEFT JOIN trap**: a filter on the right table in `WHERE` (e.g. `WHERE e.salary > 0`) turns it into an inner join; put such conditions in the `ON` clause to keep unmatched rows.

Q3. Self join: each employee with their manager's name.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

employee	manager
Asha	NULL
Ben	Asha

employee	manager
Chen	Asha
Divya	NULL
Esha	Divya
Farhan	Divya
Gita	NULL
Hari	Asha

Employees whose salary exceeds their manager's: `WHERE e.salary > m.salary` returns no rows here (no one earns more than their manager). Pairs of orders by the same customer: `FROM orders o1 JOIN orders o2 ON o1.customer_id = o2.customer_id AND o1.order_id < o2.order_id` gives (101,102), (101,105), (102,105), (103,106); the `<` removes self-pairs and mirrored duplicates.

Q4. Second highest salary (4 ways). Nth highest.

```
-- 1. subquery
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
-- 2. distinct + limit/offset
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
-- 3. window function
SELECT DISTINCT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM employees) t
    WHERE r = 2;
-- 4. correlated: N-th highest by counting distinct larger values
SELECT DISTINCT salary FROM employees e1 WHERE 1 = (SELECT COUNT(DISTINCT salary) FROM employees e2
    WHERE e2.salary > e1.salary);
```

All return **80000**. For the Nth, use `OFFSET N-1` or `r = N` (and `1 = ...` becomes `N-1 = ...`). `DISTINCT` matters when top salaries tie (Ben and Chen tie at 70000).

Q5. Subqueries: scalar, IN, EXISTS, correlated. Employees earning above their department average.

```
SELECT name, salary FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);
```

name	salary
Asha	90000
Divya	80000

A **correlated subquery** references the outer row and is evaluated per row (Engineering avg 76666.67, Sales avg 63333.33). Gita (alone in HR) is not above her own average; Hari's `dept_id` is NULL so the comparison is UNKNOWN and he is excluded. Equivalent, often faster: join to a `GROUP BY` derived table or use `AVG(salary) OVER (PARTITION BY dept_id)`.

Q6. Customers with no orders: NOT IN vs NOT EXISTS vs LEFT JOIN.

```
SELECT c.name FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id =
    c.customer_id);
SELECT c.name FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id WHERE o.order_id IS
    NULL;
SELECT name FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM orders);
```

All three return **Tom**. **NOT IN trap**: if the subquery returns even one NULL (e.g. an order with `customer_id` NULL), `x NOT IN (..., NULL)` is UNKNOWN for every row, so the query returns **zero rows**. Prefer `NOT EXISTS`. `EXISTS` stops at the first match; `IN` with a list is simple for constants.

Q7. GROUP BY problems: totals per city, repeat customers, monthly report.

```
SELECT c.city, SUM(o.amount) AS total FROM customers c JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.city ORDER BY total DESC;
```

city	total
Delhi	1450
Mumbai	800

Delhi = Ann (500+300+450) + Sara (200). Pune (Tom) is absent because of the inner join.

```
SELECT customer_id, COUNT(*) AS n FROM orders GROUP BY customer_id HAVING COUNT(*) >= 2;
```

Result: customer 1 (3 orders), customer 2 (2 orders).

```
SELECT SUBSTR(order_date, 1, 7) AS month, COUNT(*) AS n, SUM(amount) AS total   -- PostgreSQL:
    DATE_TRUNC('month', order_date)
FROM orders GROUP BY SUBSTR(order_date, 1, 7) ORDER BY month;
```

month	n	total
2026-01	1	500
2026-02	2	1000
2026-03	3	750

Conditional aggregation with `CASE`:

```
SELECT customer_id,
       SUM(CASE WHEN amount >= 400 THEN 1 ELSE 0 END) AS big,
       SUM(CASE WHEN amount <  400 THEN 1 ELSE 0 END) AS small
FROM orders GROUP BY customer_id;
```

customer_id	big	small
1	2	1
2	1	1
3	0	1

Q8. Customer with the highest total spend (with a CTE). Handling ties.

```
WITH t AS (SELECT customer_id, SUM(amount) AS s FROM orders GROUP BY customer_id)
SELECT c.name, t.s FROM t JOIN customers c USING (customer_id)
WHERE t.s = (SELECT MAX(s) FROM t);
```

Result: **Ann, 1250**. Comparing to `MAX` returns all tied customers, whereas `ORDER BY s DESC LIMIT 1` returns an arbitrary single one.

Q9. Window functions: ROW_NUMBER vs RANK vs DENSE_RANK (with ties).

Window functions compute over a set of related rows **without collapsing them** (unlike `GROUP BY`). Syntax: `func() OVER (PARTITION BY ... ORDER BY ... [frame])`.

```
SELECT name, dept_id, salary,
  ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn,
  RANK()       OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rk,
  DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dr
FROM employees WHERE dept_id IS NOT NULL;
```

name	dept_id	salary	rn	rk	dr
Asha	1	90000	1	1	1
Ben	1	70000	2	2	2
Chen	1	70000	3	2	2
Divya	2	80000	1	1	1
Farhan	2	60000	2	2	2
Esha	2	50000	3	3	3
Gita	3	55000	1	1	1

Ben and Chen tie: `ROW_NUMBER` breaks ties arbitrarily (2, 3), `RANK` gives both 2 and **skips** the next number (would give 4), `DENSE_RANK` gives both 2 with **no gaps**.

Q10. Top-N per group, e.g. the 2nd highest salary in each department.

```
SELECT dept_id, name, salary FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
  FROM employees WHERE dept_id IS NOT NULL) t
WHERE r = 2;
```

dept_id	name	salary
1	Ben	70000
1	Chen	70000
2	Farhan	60000

HR has no second salary, so no row. You cannot filter a window function directly in `WHERE`; wrap it in a subquery or CTE.

Q11. Running totals and previous-row comparison with `SUM() OVER` and `LAG`.

```
SELECT order_id, customer_id, amount,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS running,
  LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev
FROM orders ORDER BY customer_id, order_date;
```

order_id	customer_id	amount	running	prev
101	1	500	500	NULL
102	1	300	800	500
105	1	450	1250	300
103	2	700	700	NULL
106	2	100	800	700
104	3	200	200	NULL

Also useful: `LEAD` (next row), `FIRST_VALUE`, `NTILE(4)`, moving average with `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`, `SUM(x) OVER ()` for share of total.

Q12. Common interview SQL tasks (patterns).

```
-- duplicate values
SELECT salary, COUNT(*) FROM employees GROUP BY salary HAVING COUNT(*) > 1;        -- 70000 appears 2
    times
-- delete duplicates, keep lowest id (PostgreSQL)
DELETE FROM t a USING t b WHERE a.email = b.email AND a.id > b.id;
-- employees with no manager
SELECT name FROM employees WHERE manager_id IS NULL;                               -- Asha, Divya, Gita
-- department-wise highest earner (ties included)
SELECT e.* FROM employees e WHERE salary = (SELECT MAX(salary) FROM employees WHERE dept_id =
    e.dept_id);
-- odd employee ids
SELECT emp_id, name FROM employees WHERE emp_id % 2 = 1;                           -- 1, 3, 5, 7
-- customers who ordered in every month: compare COUNT(DISTINCT month) with total months
-- recursive CTE: management chain (org tree)
WITH RECURSIVE chain AS (
  SELECT emp_id, name, manager_id, 1 AS lvl FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, c.lvl + 1 FROM employees e JOIN chain c ON e.manager_id =
      c.emp_id)
SELECT * FROM chain;
```

Gaps and islands, consecutive-day logins (`date - ROW_NUMBER()` groups consecutive days), pivoting with `CASE`, and percentage of total (`amount * 100.0 / SUM(amount) OVER ()`) are also frequent.

Edge cases and gotchas

- `UNION` needs the same number and compatible types of columns.
- Self-join forgetting the `<` creates duplicate mirrored pairs.
- `GROUP BY` column not in `SELECT` is fine; the reverse is an error in strict DBs.
- `LEFT JOIN` + a `WHERE` condition on the right table = inner join.
- `COUNT(*)` after a join counts joined rows (fan-out) and can inflate sums when joining one-to-many twice.
- `RANK` vs `DENSE_RANK` difference appears only with ties.
- Window functions run after `WHERE`/`GROUP BY`/`HAVING`.
- `IN` vs `EXISTS`: usually the same plan today; `NOT IN` is the dangerous one.

Rapid-fire round

Question	Answer
Join returning unmatched left rows	LEFT JOIN
Join with no condition	CROSS JOIN (n x m rows)
ROW_NUMBER with tie	Unique numbers, arbitrary tie order
RANK with tie	Same rank, skips next
DENSE_RANK with tie	Same rank, no skip
Safe anti-join	NOT EXISTS or LEFT JOIN ... IS NULL
Correlated subquery	Re-evaluated per outer row
CTE keyword for loops over hierarchy	WITH RECURSIVE

NoSQL, CAP and Scaling Concepts

Short but commonly asked: SQL vs NoSQL, the four NoSQL families, CAP, BASE, replication/sharding, and when to choose what.

Q1. SQL vs NoSQL. When to choose which?

	SQL (relational)	NoSQL
Schema	Fixed, enforced	Flexible / schemaless
Relationships	Joins, foreign keys	Embedded or application-managed
Transactions	Strong ACID, multi-row	Often single-record/limited (some support more)
Scaling	Vertical, read replicas, sharding harder	Designed for horizontal scale
Query	Powerful ad hoc SQL	Access-pattern-driven

Choose SQL for structured data, relationships, reporting, strong consistency (payments, inventory). Choose NoSQL for huge scale, flexible/evolving data, simple access patterns, low-latency key lookups. Many systems use both (polyglot persistence).

Q2. Types of NoSQL stores.

Key-value (Redis, DynamoDB): fastest lookups by key; **document** (MongoDB, Couchbase): JSON documents with rich queries; **wide-column** (Cassandra, HBase): huge write-heavy distributed data; **graph** (Neo4j): relationships first (social networks, recommendations). Also search engines (Elasticsearch) and time-series databases.

Q3. CAP theorem and BASE vs ACID.

In a distributed data store you can guarantee at most two of **Consistency** (every read sees the latest write), **Availability** (every request gets a non-error response), **Partition tolerance** (system works despite network splits). Since partitions happen, the real choice during a partition is **CP** (refuse some requests to stay consistent: HBase, MongoDB defaults, Redis cluster) or **AP** (answer, possibly stale: Cassandra, DynamoDB eventual). **PACELC** adds: else (no partition) latency vs consistency. **BASE**: Basically Available, Soft state, Eventually consistent (the NoSQL alternative to ACID).

Q4. Sharding strategies, consistent hashing, replication, and caching in one view.

Sharding: **range** (simple, hot spots), **hash** (even, no range scans), **directory/lookup**. **Consistent hashing** maps keys and nodes on a ring so adding/removing a node moves only about 1/n of the keys. Replication: leader-follower (read scaling, failover, replica lag), multi-leader, leaderless with quorums (R + W > N gives strong reads). Caching: **cache-aside** (read through cache, fill on miss), write-through, write-back; invalidation is the hard part; TTLs bound staleness.

Edge cases and gotchas

- "NoSQL is faster" is not a rule: it depends on the access pattern.
- Eventual consistency means readers may see stale data briefly.
- Hash sharding loses efficient range queries.
- Adding a shard with plain `key % n` remaps almost all keys (consistent hashing avoids this).
- Joins across shards are expensive; denormalise or co-locate related data.

Rapid-fire round

Question	Answer
CAP letters	Consistency, Availability, Partition tolerance
Cassandra is typically	AP

Question	Answer
BASE vs ACID	Eventual consistency vs strong guarantees
Quorum rule	R + W > N
Document store example	MongoDB
Graph store example	Neo4j
Consistent hashing benefit	Minimal key movement on resize